

September 13, 2007

Status
Final

Revision
V1.0

Document File Name
*Bluetooth® User Interface
Flow Diagrams for Bluetooth
Secure Simple Pairing
Devices*

Document Owner
Usability Expert Group

E-mail Address
usability-
main@bluetooth.org



***BLUETOOTH®* USER INTERFACE FLOW DIAGRAMS FOR *BLUETOOTH* SECURE SIMPLE PAIRING DEVICES**

Usability Expert Group

ABSTRACT: This document contains User Interface Flow recommendations for *Bluetooth* pairing including Secure Simple Pairing.



Revision History		
Revision	Date	Description
0.0.1	August 16, 2006	Initial Draft
0.0.2	August 29, 2006	Added Background task flows and corrected UI flows found during meeting.
0.0.3	September 5, 2006	Added REJECT, SEND/RECEIVE Task and UI flow. Updated UI flows found during meeting.
0.0.4	September 8, 2006	Updated UI / Task flows based on feedback from UEG group.
0.0.5	September 21, 2006	Updated UI / Task flows and document based on feedback from UEG Face to Face Meeting.
0.0.6	September 22, 2006	Updated UI / Task flows and document based on feedback from UEG Face to Face Meeting.
0.0.7	September 27, 2006	Updated UI / Task flows and document based on feedback from UEG Face to Face Meeting.
0.0.8	September 28, 2006	Incorporated into Bluetooth SIG template. Added Out Of Band Task and User Flow. Closed open items from weekly call.
0.0.9	October 4, 2006	Prepared for BARB submission. Updated with tasks called out from weekly meeting. Shrunk embedded pictures to reduce document storage size.
D05r10	October 5, 2006	Edited for format and distribution as D05
D05r11	October 9, 2006	Edited for format and distribution as D05
D05r12	October 12, 2006	Added Out Of Band Task and User Flow in document
D05r13	October 18, 2006	Updated the Out Of Band Task and User Flow information to identify other Out Of Band technologies and to make the Out Of Band User Flow technology agnostic.
D05r14	October 19, 2006	Updated with initial BARB comments received
D05r15	November 1, 2006	Updated with initial BARB comments received. Added corrected Out Of Band flows
D05r16	November 29, 2006	Updated with marketing comments received.
D05r17	December 13, 2006	Updated with OOB structural changes.
D05r18	January 3, 2007	Added OOB discussion items from the call.
D05r19	January 17, 2007	Updated OOB discussion items and cleaned-up the remainder of the document.
D09_r00	January 18, 2007	Final edits and publishing for review.
D09_r01	January 29, 2007	Feedback with CSWG comments
D09_r02	February 1, 2007	Feedback with UEG/CSWG comments
D09_r03	February 9, 2007	Feedback with UEG/CSWG comments



D09_r04	February 16, 2007	Feedback with UEG/CSWG comments
D09_r05	February 21, 2007	Feedback with UEG/CSWG comments
D09_r06	March 2, 2007	Updated with new UI flow removing “verified”
D09_r07	March 21, 2007	Updated with BARB comments
D09_r08	April 2, 2007	Updated with new State Diagram
D09_r09	May 30, 2007	Updated for GAP changes to Secure Simple Pairing
D09_r10	June 19, 2007	Include comments from Paul and Frank
D09_r11	July 13, 2007	Include comments from BARB, CWG, and UEG groups
D09_r12	July 16, 2007	Editorial review
D09_r13	July 24, 2007	Editorial review with UI and task flow redefinition and addition of bonding, trust, authentication and authorization sections.
V1.0	August 9, 2007	Approved for publication.

Contributors	
Name	Company
Ryan Seick	Motorola
Peter Hauser	Bluetooth SIG
Rob Franzo	Broadcom
Frank Armstrong	Kodak
Yeesin Chan	Sprint
Niall Gillespie	CSR
Kimberly Saucedo	Plantronics
Jun Shen	AT&T
Harry Zhang	Motorola
Paul Wootten	TMTI
Emma Oivio	Nokia
Riitta Peltonen	Nokia
Anders Schroeder	Nokia
Dustin Voss	Open Air Interface



DISCLAIMER AND COPYRIGHT NOTICE

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NONINFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE. Any liability, including liability for infringement of any proprietary rights, relating to use of information in this document is disclaimed. No license, express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

This document is for comment only and is subject to change without notice.

Copyright © 2001–2007 Bluetooth® SIG, Inc. All copyrights in the *Bluetooth* Specifications themselves are owned by Ericsson AB, Lenovo, Intel Corporation, Microsoft Corporation, Motorola, Inc., Nokia Corporation, and Toshiba Corporation. *Other third-party brands and names are the property of their respective owners.



CONTENTS

1	FOREWORD.....	8
2	SCOPE.....	9
3	REFERENCES.....	10
4	TASK FLOW OVERVIEW	11
5	DESIGN CONSIDERATIONS.....	13
	5.1 LENGTH OF DISCOVERABILITY (TIMEOUT)	13
	5.2 AMOUNT OF THOUGHT REQUIRED.....	13
	5.2.1 MEMORIZATION	13
	5.2.2 NUMBER OF DIGITS.....	13
	5.2.3 CREATIVE THOUGHT	13
	5.2.4 UNIQUENESS.....	14
	5.2.5 AUTOMATION	14
	5.3 FEEDBACK TO USERS (HELP)	14
	5.4 NUMBER OF KEY PRESSES (2 MAXIMUM)	14
	5.5 ALWAYS USE ENCRYPTION.....	14
	5.6 RECONNECTION PERFORMANCE	14
	5.7 CAN THE USER BE BLIND, DEAF, AND/OR ILLITERATE?	14
	5.8 DOES THIS IMPLEMENTATION CAUSE OTHER USER ISSUES?	14
6	TRUST, AUTHORIZATION, AND AUTHENTICATION	15
	6.1 FROM A USER PERSPECTIVE	15
	6.2 FROM AN IMPLEMENTERS PERSPECTIVE.....	15
	6.3 WARNING ON “BLANKET” TRUSTING ALL SERVICES.....	15
7	PAIRING AND BONDING	17
	7.1 SHOULD WE STORE THE LINK KEY OR NOT?	17
	7.2 MAN IN THE MIDDLE PROTECTION	17
	7.3 DEDICATED VERSUS GENERAL BONDING	17
	7.4 NON-BONDABLE MODE	17
	7.5 NON-BONDABLE PROFILE EXAMPLES	18
8	TASK FLOWS	19
	8.1 FACTORY DEFAULT FLOW.....	19
	8.2 ADD DEVICE	19
	8.2.1 JUST WORKS TASK FLOW	20
	8.2.2 NUMERIC COMPARISON TASK FLOW	21
	8.2.3 PASSKEY ENTRY TASK FLOW.....	22
	8.2.4 LEGACY PAIRING TASK FLOW	22
	8.3 ADD SERVICE	24
	8.3.1 ADD SERVICE TASK FLOW	24
	8.4 CONNECT TASK FLOW	25
	8.5 SEND / RECEIVE TASK FLOW	25
	8.6 REJECT / BLOCK CONNECTION FLOW.....	27
	8.7 OUT OF BAND TASK FLOW	27



	8.7.1	BIDIRECTIONAL TASK FLOW.....	28
	8.7.2	UNIDIRECTIONAL TASK FLOW	29
	8.7.3	ADD DEVICE TASK FLOW.....	29
	8.7.4	ACTION SPECIFIC TASK FLOW.....	31
9		USER INTERFACE FLOWS	32
	9.1	FACTORY DEFAULT USER INTERFACE FLOW.....	32
	9.2	ADD DEVICE	32
	9.2.1	JUST WORKS UI FLOW	32
	9.2.1.1	MOBILE PHONE TO HEADSET.....	33
	9.2.1.2	PC TO PDA	34
	9.2.1.3	V-CARD TRANSFER	35
	9.2.1.4	V-CARD TRANSFER –OPTIMIZED.....	37
	9.2.2	NUMERIC COMPARISON UI FLOW.....	39
	9.2.2.1	PC TO MOBILE PHONE	39
	9.2.2.2	SEND FILE - MOBILE PHONE TO MOBILE PHONE	41
	9.2.3	NUMERIC [PASSKEY] ENTRY AND LEGACY PAIRING UI FLOW	43
	9.2.3.1	DEVICES WITH A HID KEYBOARD	44
	9.2.3.2	LEGACY HID PASSKEY PAIRING WITH A PC.....	45
	9.2.4	FIXED PIN PAIRING FOR DEVICES WITH A HEADSET	46
	9.2.5	OUT OF BAND UI FLOWS.....	47
	9.2.5.1	ADD DEVICE – USING NFC (CAMERA TO PRINTER).....	49
	9.2.5.2	UNIDIRECTIONAL PDA TO PDA	50
	9.2.5.3	BIDIRECTIONAL PDA TO PDA	51
	9.2.5.4	SEND FILE – USING NFC FROM PC TO MUSIC PLAYER	52
	9.3	CONNECTION REJECTION FROM PC TO PDA	53
10		LISTS	54
	10.1	DEVICE LIST	54
	10.2	TRUSTED DEVICE LIST	54
	10.3	BLOCKED DEVICE LIST.....	54



LIST OF FIGURES

FIGURE 1: TASK FLOW EXAMPLE	11
FIGURE 2: SIMPLIFIED CONNECT STATE DIAGRAM.....	12
FIGURE 3: FACTORY DEFAULT TASK FLOW	19
FIGURE 4: "JUST WORKS" TASK FLOW	20
FIGURE 5: NUMERIC COMPARISON TASK FLOW.....	21
FIGURE 6: PASSKEY ENTRY TASK FLOW	22
FIGURE 7: LEGACY PAIRING TASK FLOW	23
FIGURE 8: ADD SERVICE TASK FLOW	24
FIGURE 9: CONNECT TASK FLOW.....	25
FIGURE 10: SEND / RECEIVE TASK FLOW	26
FIGURE 11: REJECT CONNECTION TASK FLOW	27
FIGURE 12: OOB BIDIRECTIONAL TASK FLOW	28
FIGURE 13: OOB UNIDIRECTIONAL TASK FLOW	29
FIGURE 14: OOB "ADD DEVICE" TASK FLOW	30
FIGURE 15: OOB "ACTION SPECIFIC" TASK FLOW	31
FIGURE 16: "JUST WORKS" HEADSET PAIRING USER INTERFACE FLOW	33
FIGURE 17: ADD DEVICE - "JUST WORKS" UI FLOW.....	34
FIGURE 18: V-CARD TRANSFER – STANDARD "JUST WORKS" UI FLOW	36
FIGURE 19: V-CARD TRANSFER - OPTIMIZED "JUST WORKS" UI FLOW	38
FIGURE 20 ADD DEVICE - NUMERIC COMPARISON USER INTERFACE FLOW	40
FIGURE 21: NUMERIC COMPARISON - SEND FILE (PICTURE) USER INTERFACE FLOW	42
FIGURE 22: SECURE SIMPLE PAIRING FOR DEVICES WITH A HID KEYBOARD UI FLOW.....	44
FIGURE 23: LEGACY HID PASSKEY PAIRING WITH A PC USER INTERFACE FLOW	45
FIGURE 24: FIXED PIN PAIRING FOR DEVICES WITH A HEADSET USER INTERFACE FLOW	46
FIGURE 25: PC TO MOBILE PHONE OOB USER INTERFACE FLOW	48
FIGURE 26: CAMERA TO PRINTER NFC USER INTERFACE FLOW	49
FIGURE 27: PDA TO PDA UNIDIRECTIONAL OOB USER INTERFACE FLOW	50
FIGURE 28: OUT OF BAND – BIDIRECTIONAL PDA TO PDA USER INTERFACE FLOW.....	51
FIGURE 29: SEND FILE – OOB USING NFC FROM PC TO MUSIC PLAYER UI FLOW	52



1 Foreword

Consistent and simple to use interfaces lead to the adoption of new technology. *Bluetooth* wireless technology faces unique user experience challenges as products with a wide variation of end uses and human interfaces enter the market. Some products will offer a rich varied user interface; other more simplified devices may offer none.

Bluetooth technology is designed to be usable without the benefit of physical cues to provide feedback to the end user on status of the task being accomplished. Furthermore *Bluetooth* technology enabled devices must be designed for peoples of many languages and cultures. Thus additional efforts are needed to ensure that common *Bluetooth* technology terms and user actions are implemented consistently between manufacturers.

This document provides a straightforward set of recommendations for user interface designers when designing the *Bluetooth* Secure Simple Pairing experience for their respective applications.

NOTE: If there is a discrepancy between the definitions and procedures within this document and the underlying *Bluetooth* specifications, the *Bluetooth* specifications override any definition or procedure herein.



2 Scope

The following flows summarize a set of step-by-step user actions for *Bluetooth* pairing including the Secure Simple Pairing feature introduced in the Bluetooth Core Specification version 2.1 + EDR. The flows provided are not exhaustive and do not cover all use cases. This document is meant to provide UI implementers with a solid set of examples to suggest some common implementations.

These user interface flows are not specifications; however they are guidelines on how to achieve a uniform *Bluetooth* pairing experience across devices. Developers are encouraged to use these user interface flows to provide consistency throughout their user interfaces.

The Usability Expert Group recognizes the concern that having multiple different “Out Of Band” (OOB) mechanisms present on a single class of device may fragment and ultimately reduce the usability in that device's market. It is the UEG recommendation that a single OOB mechanism be used across a single device class.

Unless otherwise stated in the document and/or diagrams, all flows pertain to the *Bluetooth* interface and not to other technologies that may be present on any given device.

The user interface flows dealing with authenticated device reconnections are outside the scope of this document.

It is out of the scope of this document to define how to put a device into Factory Default¹. This Factory Default only applies to the state of the *Bluetooth* technology not for the entire device.

¹ The initialized condition for the Bluetooth setting of a Device.



3 References

- [1] *Bluetooth* Secure Simple Pairing User Terminology
To Be added when Final Approval and document location are completed
- [2] *Bluetooth* Secure Simple Pairing Usability Metric Whitepaper
https://programs.Bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=43654
- [3] *Bluetooth* Secure Simple Pairing Whitepaper
https://programs.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=41652
- [4] *Bluetooth* v2.1 + EDR Draft v1.0 Core Specification
https://programs.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=66471
- [5] *Bluetooth* v2.0 + EDR Core Specification
https://programs.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=40560
- [6] *Bluetooth* v1.2 Core Specification
https://programs.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=14492



4 Task Flow Overview

Task flows show the state to state transitions and show what is exposed to the User Interface. The following example shows how to read the task flows. If there is no line present and the action is in the middle, one or both devices may perform the same actions. If there is a line dividing the roles there are independent actions taking place on each device.

A device with Bluetooth service enabled is capable of initiating or accepting connections. The Bluetooth connections are used to initiate pairing to an authenticated device. A device should not allow connections to a device if listed in the blocked device list.

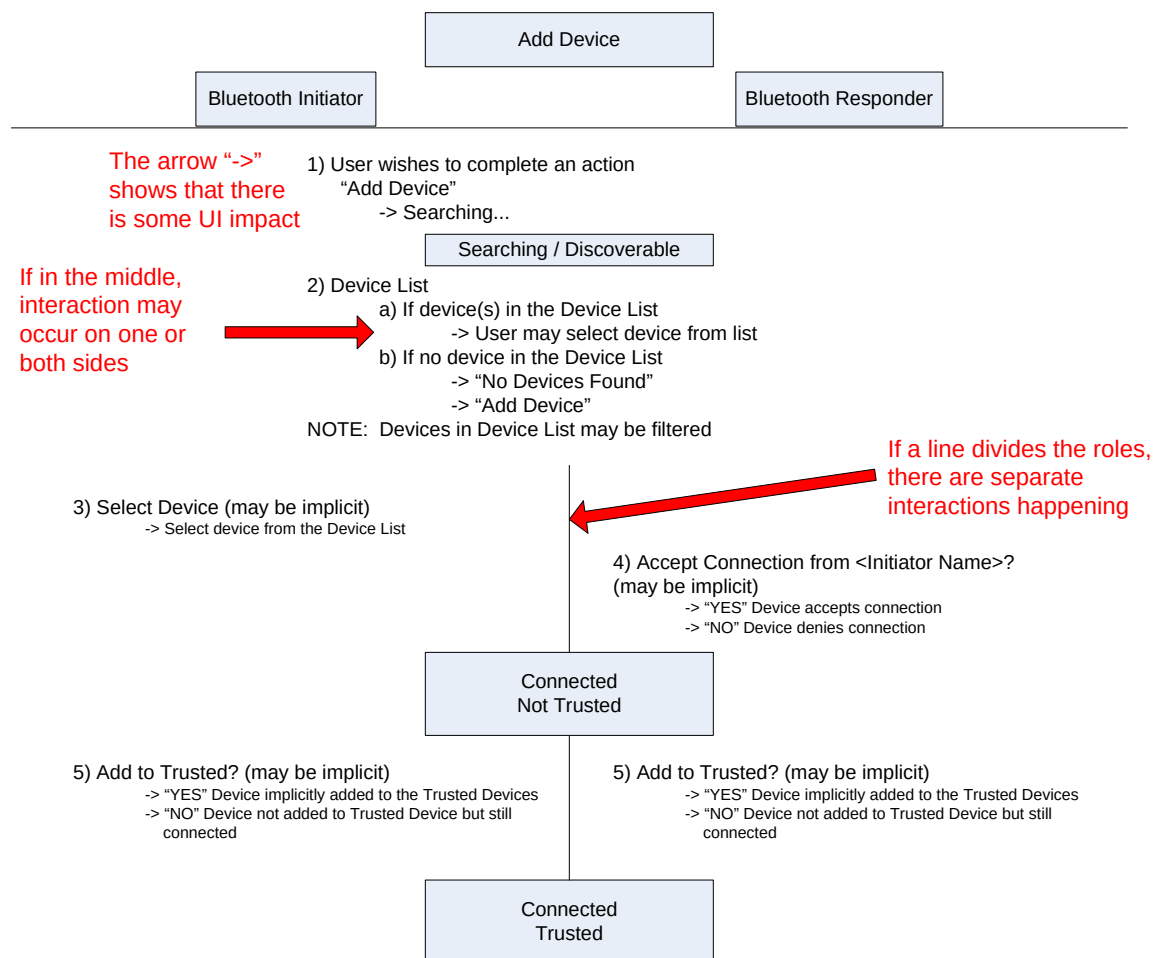


Figure 1: Task Flow Example

The flow is shown in a top to bottom sequence with each numeric value as a sequenced step with increasing numeric values showing the timed steps. Sequences with the same numeric value are actions that may happen simultaneously. "Options" within a sequence step are shown with alphabetical values. Arrows "->" are shown as suggested feedback to the user via the User Interface.

Where a means of text display is available, both devices should request the Device Name of the other device. The Device Name is then displayed to the user in the Device List and in prompts that display the Bluetooth Initiator or Bluetooth Responder Device Name instead of the Device Address.

Upon completing the *Bluetooth* pairing process, the manufacturer may wish to enable the user to "Trust" the device and services. In this event, the device is added to the user's Trusted Devices list and the said device



will be allowed to reconnect. "Trust" may apply for specific services or the entire device not requiring the said device to be authenticated. If the authentication level is insufficient for access to certain services, then the user may be asked to re-authenticate the device before the said services can be exposed.

The following diagram illustrates the connection states from a user's point of view that apply to devices and the transitions between these states:

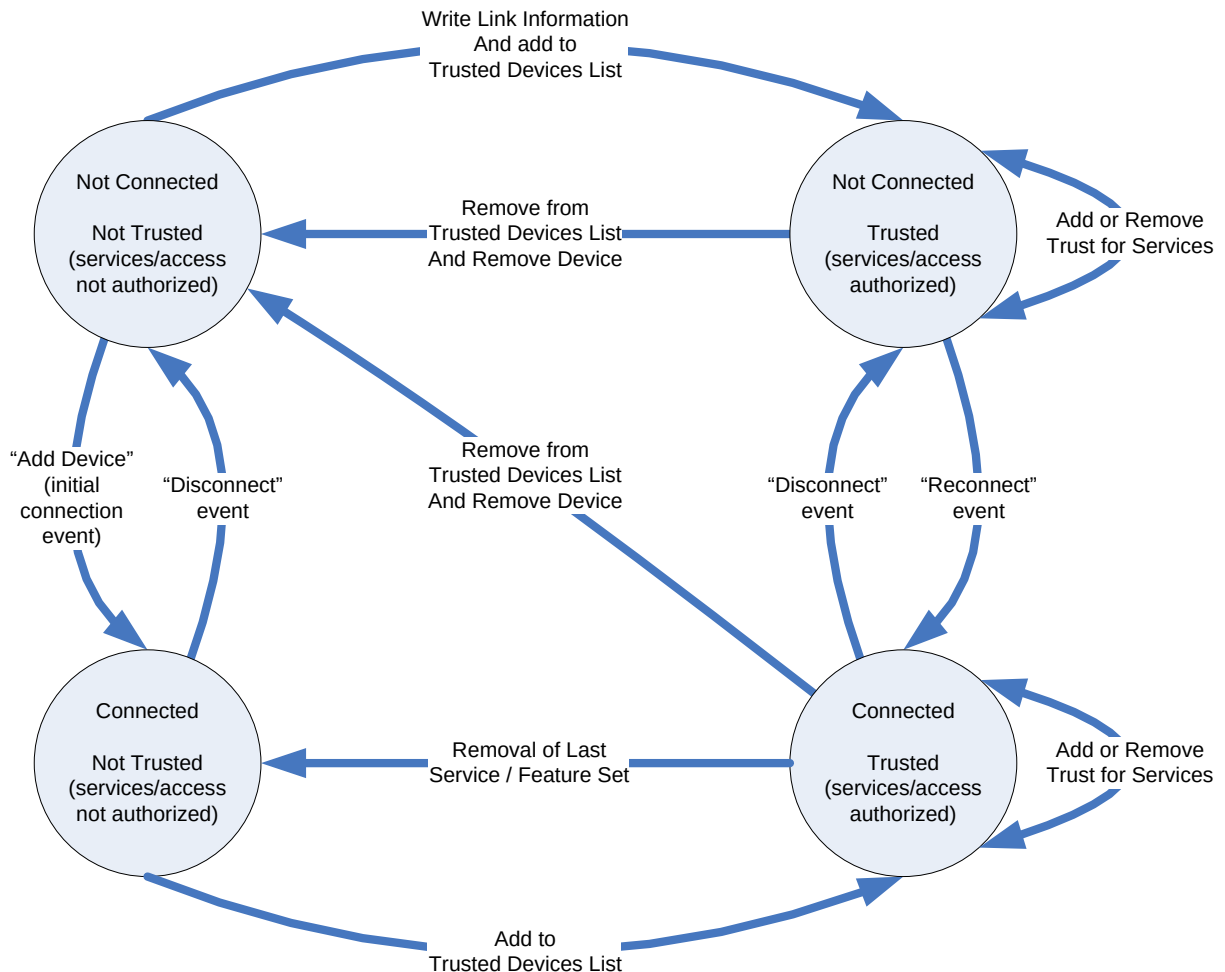


Figure 2: Simplified Connect State Diagram

As a result of the mandatory security features enabled in *Bluetooth v2.1+EDR*, implementers need to be aware of the implications of requiring encrypted connections for all *Bluetooth v2.1+EDR* devices. One such implication is that devices may require extra memory to store authenticated devices (link keys, BD_ADDRs plus other information about the device). In cases where many devices may be forming connections with a given target device, more memory may be necessary to ensure that new devices can be added to the list of trusted devices on the target device.

Furthermore, buffers that delete the oldest device in the trusted devices list may be preferred over other buffering mechanisms to ensure that when the memory is full, new devices may still be added without explicitly clearing the memory on the target device. NOTE: The issue of verifying the identity (authenticating) of the user/device is one that is not illustrated here. Devices may be considered "Trusted" and thus authorized to use certain services even if the device has not undergone an identity verification process. The manufacturer is, however, cautioned to limit access to use cases that require a level of authentication that is sufficient for the service level access provided.



5 Design Considerations

Design considerations are general guidelines that should be used when architecting user experiences. These design considerations help ensure that certain *Bluetooth* technology usability principles are followed so as to improve the overall *Bluetooth* user experience. To understand the rationale behind these design considerations please refer the Secure Simple Pairing Metrics documents reference [2].

5.1 LENGTH OF DISCOVERABILITY (TIMEOUT)

The length of the discoverability timeout should be set so as to ensure that the user has enough time to determine which steps are needed to accomplish the task (i.e. pairing a phone to a headset). The Length of Discoverability should be long enough such that the user has plenty of time to initiate and ultimately complete the process of adding a *Bluetooth* technology enabled device supporting Secure Simple Pairing or the legacy pairing mechanism. Each manufacturer is encouraged to run studies as defined in the Bluetooth UEG Metric Whitepaper [2] to identify the Overall Task Time for each given pairing mechanism and to set the Length of Discoverability to a multiple of the longest Overall Task Time identified by this analysis. A general rule is to set the Length of Discoverability to at least three times (3x) the longest Overall Task Time.²

5.2 AMOUNT OF THOUGHT REQUIRED

One of the tenets of good usability is not to require a lot of thought from the user. In other words, the best user interfaces are intuitive enough such that no explanation is required to enable most users to use the interface effectively. These interfaces use a mix of visual, auditory, and other cues to help organize functions and features in a logical manner. The following design considerations can be used to evaluate the thought needed to navigate a given User Interface.

5.2.1 MEMORIZATION

Do users need to memorize anything even in the short term?

In general, if a user needs to memorize anything, then they will be far more likely to make errors. Therefore, the less you make users memorize, the more easily they will be able to perform tasks.

5.2.2 NUMBER OF DIGITS

How many digits must be entered by the user to complete the process?

Studies have shown that users have significant trouble when trying to remember more than 7 digits ([Miller, G. \(1956\): "The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information", *Psychological Review*, vol. 63 pp. 81-97 \[1\]](#)). The Usability Expert Group recommends that *Bluetooth* Secure Simple Pairing with any association model that uses 6 digits be split into two groups of three separated by a single space when displayed to help minimize the usability impacts caused by the numeric entry length.

5.2.3 CREATIVE THOUGHT

Do users need to "invent" keys, codes, names, etc.?

Whenever a user is asked to invent a number, the user will typically revert back to a previously memorized number. This can lead to security vulnerabilities when the number in question is used as a random value to help establish security. Furthermore, users are limited as to how many numbers they can invent and then recall later. The fewer things a user needs to "invent", the less the user has to think about the user interface and thus can more rapidly accomplish his/her task. *Bluetooth* Secure Simple Pairing does not require the user to invent a PIN as was the case in the legacy pairing method.

² Refer to bonding in the GAP profile



5.2.4 UNIQUENESS

Does the user need to guarantee the uniqueness of invented data?

Users tend to select numbers and/or codes and phrases that they have heard, seen, or used previously. As a result, asking a user to guarantee the uniqueness of the invented data is a very challenging proposition for most users and requires extensive thought as the user sifts through memories in search of a unique number, code, and/or name.

5.2.5 AUTOMATION

How many steps can be automated and/or require only Y/N answers?

The more steps that can be automated inline with the user's expected responses and/or require only Yes and No answers each of which requires minimal thought, the less the user is forced to think about each step.

5.3 FEEDBACK TO USERS (HELP)

Help functions and/or guidance in the form of a user manual, quick start guides, stickers, or device cues should be provided whenever user confusion exists and the source of the confusion cannot be overcome through careful redesign. Even in cases where all steps have been taken to avoid confusion, such features aid those users who prefer reading a manual ahead of time.

5.4 NUMBER OF KEY PRESSES (2 MAXIMUM)

Some suggest that if a menu item is more than 2 key presses deep it is far less likely to be used. In the case of *Bluetooth* Secure Simple Pairing, users are unlikely to use the *Bluetooth* capability if embedded by more than two layers in the user interface. Implementers are thus encouraged to keep *Bluetooth* pairing functions at either the top level or to initiate connections between *Bluetooth* devices based on user actions such as sending a picture or listening to a song.

5.5 ALWAYS USE ENCRYPTION

By definition, those products that implement the *Bluetooth* V 2.1 + EDR specification use encryption when forming connections with other *Bluetooth* devices that implement the *Bluetooth* V 2.1 + EDR specification. When connecting with legacy *Bluetooth* technology enabled devices, the decision to use or not to use encryption is based on the profile and/or product with which the connection is being established, and in some cases based on user choice.

5.6 RECONNECTION PERFORMANCE

Device implementers are encouraged to minimize the time it takes to reconnect one device with another after an unexpected connection loss. In the event of connection loss, implementers are encouraged to critically evaluate their implementations for corner cases where neither device reconnects or where both devices attempt to reconnect at the same time. This could lead to a poor user experience.

5.7 CAN THE USER BE BLIND, DEAF, AND/OR ILLITERATE?

User interface designers are encouraged to think carefully about their implementations and the demographic that they are targeting with their product to ensure that users with disabilities or who do not know the language can still use the primary device functions.

5.8 DOES THIS IMPLEMENTATION CAUSE OTHER USER ISSUES?

It is important to analyze the dependencies between parts of a given implementation that may have adverse and/or unexpected impacts on other parts of the implementation. For example, if a user adds a mobile device, then the impact of the newly added mobile device on the rest of the implementation should be understood. Failing to understand these effects can lead to surprises which confuse users and/or erode confidence in the product.



6 Trust, Authorization, and Authentication

This section addresses the concept of “Trust” and how it applies to Authorization and *Bluetooth* Authentication as described in the *Bluetooth* v2.1+EDR Core Specification of reference [4]

6.1 FROM A USER PERSPECTIVE

A user can enable a trust relationship with another device by indicating that the user has confidence in being able to carry out certain (or all) services, features or applications with that other device. Trust can be explicitly or implicitly established during the Secure Simple Pairing process through UI functions that ask the user to authorize the use of certain services, features, or applications. Trust may be associated with the device as a whole or with a subset of its functionality (e.g. a Service) and is conceptually presented to the user on an individual device, service, feature, or application basis. A user can choose to refuse to trust a device thereby not establishing a Trust relationship. In doing so, the user would require notification and confirmation should the device attempt to connect or to use any services, features, or applications in the future.

6.2 FROM AN IMPLEMENTERS PERSPECTIVE

Users may be notified that a previously paired device would like to connect if the BD_ADDR and link key was stored. The Bluetooth authentication mechanism ensures that only mutually authenticated devices can reconnect. Trusted devices can thus be authenticated and user notification and confirmation is therefore not mandatory for trusted devices.

An implementer must be able to manage Trust relationships within the device. Trust can be associated with devices and/or services. Services include profiles, and features within profiles. Note that this indicates to the implementer that an information table must be created within the device to maintain aspects of the Trust relationships (i.e. authenticated, not authenticated, device information, services, etc.).

For example, a mobile phone and a headset can be authenticated with the 'just works' association model and be implicitly granted complete device trust for all available services on the device. An implementer made the decision that man in the middle protection was not necessary.

However, more complicated devices may establish a trusted device relationship, yet not trust (authorize) access to certain services. It is the responsibility of the device implementer to determine the extent of the access that can be granted given the level of device authentication and the available services. For example, a phone can establish a trusted device relationship with a PDA that includes services for transferring a v-Card. Yet other services such as the File Transfer Profile may require explicit trust to be granted by the user or may not be available if 'just works' was used for establishing the initial device relationship. In this case, one or more services can be added to the trusted list if desired by the user OR the user may be required to explicitly authenticate the device again before trust can be granted.

6.3 WARNING ON “BLANKET” TRUSTING ALL SERVICES

Giving blanket trust to a device (as implied by the “trusted device list”) is dangerous and it may lead to products with security holes. Implementers need to take this to the granularity of the services or applications that a given device will be permitted to use except under much understood usage conditions.

One problem is assuming that a particular type of device supports only one service. For example the UI designer might assume that a mono-headset could be trusted because it supports only HFP. On the other hand a hacker could have a headset (car kit or PDA) that also implements FTP and will download files. UI designers should generally not trust a device implicitly and rather only trust the services or profiles on a device to which access was initially granted. It does not necessarily stop at the level of profiles either, because allowing a headset to make calls may need to be managed. For instance, an HF device may not be allowed to make international calls, only domestic, or only local calls.



When devices are being added to the trusted device list it is the UI designer's responsibility to ask the necessary questions and to properly limit the scope of that trust. In general, trust should be assigned only at the level of service(s) or function(s) for that particular device. If the device being accepted into the trusted device list is a personal device it may be acceptable to give "blanket" approval or at least at the level of the currently defined profiles or functions of that device. If the device is that of a friend or acquaintance the configuring of "trust" should involve some questions to the user about what services are to be accepted for that device.



7 Pairing and Bonding

The *Bluetooth* v2.1+ EDR Core Specification handles pairing, general bonding, and dedicated bonding in different ways as they serve different purposes. Once the bonding process has completed, the expectation is that both sides of the Bluetooth link have stored the other device's connection information in non-volatile storage. In doing so, the other device may reconnect without going through the pairing and bonding process again. The intent to "Bond" is negotiated at the start of the connection establishment process for *Bluetooth* v2.1 + EDR devices, unlike the *Bluetooth* v2.0 + EDR and prior specifications.

7.1 SHOULD WE STORE THE LINK KEY OR NOT?

Bonding is defined as the connection process where the link key and connection information are stored in non-volatile memory. Thus bonding implies that the device's link information is available after the current connection ceases. If the link key and current connection information are not stored, then the connection process uses "Pairing". The user might be given an option to decide whether or not bonding should be used during or before the pairing process begins (e.g. the user might be presented with a menu with the choices):

- a) Accept connection and add device to trusted list
- b) Accept connection, but do not add device to trusted list
- c) Reject connection

Alternatively, the implementation could use 2 different decision screens: one to accept/reject the connection and another to add the device to the trusted devices list. This allows the user to keep the decisions separate. For some profiles and/or devices, the decision to perform bonding or not may be implicit (e.g. a headset would typically request Bonding during connection establishment)

7.2 MAN IN THE MIDDLE PROTECTION

Requiring Man In The Middle (MITM) protection should be a rare thing. MITM is provided for cases where explicit user authentication is truly needed, for example by government organizations, or by profiles such as the HID and SIM Access Profile that allow access to personal and/or mission critical data.

7.3 DEDICATED VERSUS GENERAL BONDING

User should never be exposed to terms such as Dedicated Bonding or General Bonding though the action of "bonding" may be going on in the background. The UI, however, may be impacted by the differences in the process of bonding vs. pairing. Dedicated bonding is generally associated with the "Add Device" user action where no particular service is being selected prior to initiating the connection. General Bonding is more often initiated from a "context menu" which inherently specifies a service or action (e.g. a menu option to add a mouse, add a stereo headset, and send a file).

For single-function devices, such as A2DP headsets [that do not include HFP support], the Add Device process may use General Bonding. By using General Bonding, the service level connection may be made as soon as the Bonding process completes. A2DP headsets on a PC, for example, are often treated like speakers, and sound begins immediately streaming once the device has been added.

7.4 NON-BONDABLE MODE

Implementers may choose to add a "non-Bondable" mode to their devices. The primary use cases for the "non-Bondable" mode are:

- Public services that do not require bonding (e.g. kiosks, ATM machines, public access points, public photo printers)



- On devices where the user wishes to be visible but not connectable, typically the “non-Bondable” mode would be listed as an advanced option and a user friendly term such as “not connectable” would be selected.

7.5 NON-BONDABLE PROFILE EXAMPLES

A business card exchange or potentially a file transfer does not require the use of a "non-Bondable" mode and Bonding may still occur for other profiles and services that require bonding on the given device. Bondable mode, Limited Discoverable mode, and Connectable mode should be available options on devices when using the "Find me" mode.



8 Task Flows

8.1 FACTORY DEFAULT FLOW

This is the description of the task flow in which one or more of the devices are in factory default³. The factory default flow assumes that the user has either just purchased or fully reset their device(s). A mechanism should exist to put devices into the factory default state. The flow starts with the factory default and extends to the point where the user is ready to use *Bluetooth* on their device(s). A Factory Default device may power up in three different states (Single Function – Single I/O, Single Function – Complex I/O, and Multi-Function as shown below.

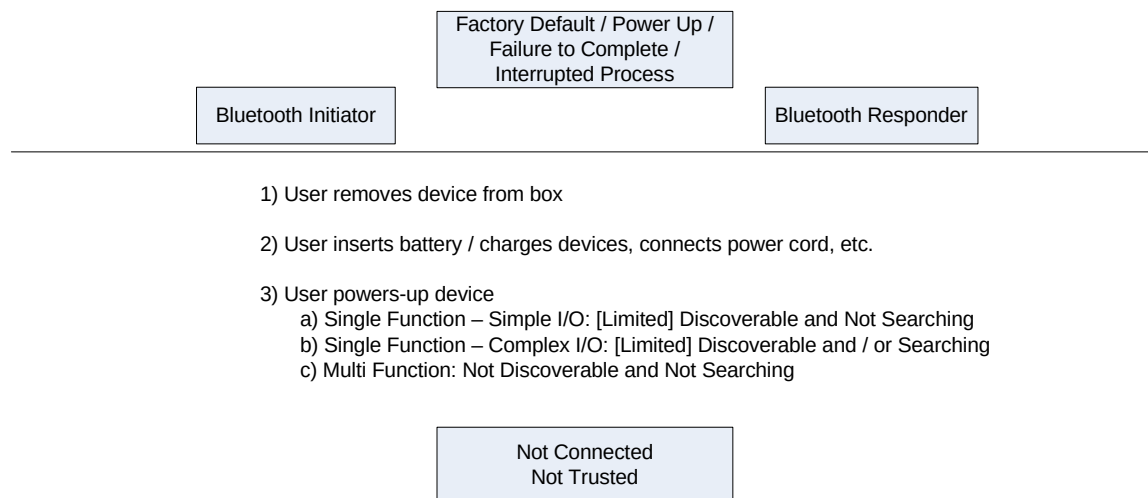


Figure 3: Factory Default Task Flow

8.2 ADD DEVICE

This is used when connecting to a device that is not in the Trusted Device List. When adding a device, a certain level of trust is implied for the given device since the user is adding the device without context⁴. The default level of trust is that the device may reconnect without requiring user confirmation though the device should not have access to any service without trust being granted explicitly for the given service. The exception is for devices that have only a single service such as a Human Interface Device or a Hands-Free device.

³ Defined in reference Bluetooth Secure Simple Pairing Lexicon [1]

⁴ Refer to bonding as described in the GAP Profile



8.2.1 JUST WORKS TASK FLOW

The “Just Works” task flow is the *Bluetooth* Secure Simple Pairing mechanism that requires no further user action such as a [Numeric] Passkey Entry, Numeric Comparison, or OOB association models. The “Just Works” task flow is needed when one or both devices do not have a User Interface capable of displaying or entering a number or passkey or passing information via an OOB channel but is also useful whenever product implementers want to make the user experience easier and have accepted the increased risk of security attacks. The Bluetooth Initiator and Bluetooth Responder are in Searching / Discoverable state until roles are defined.

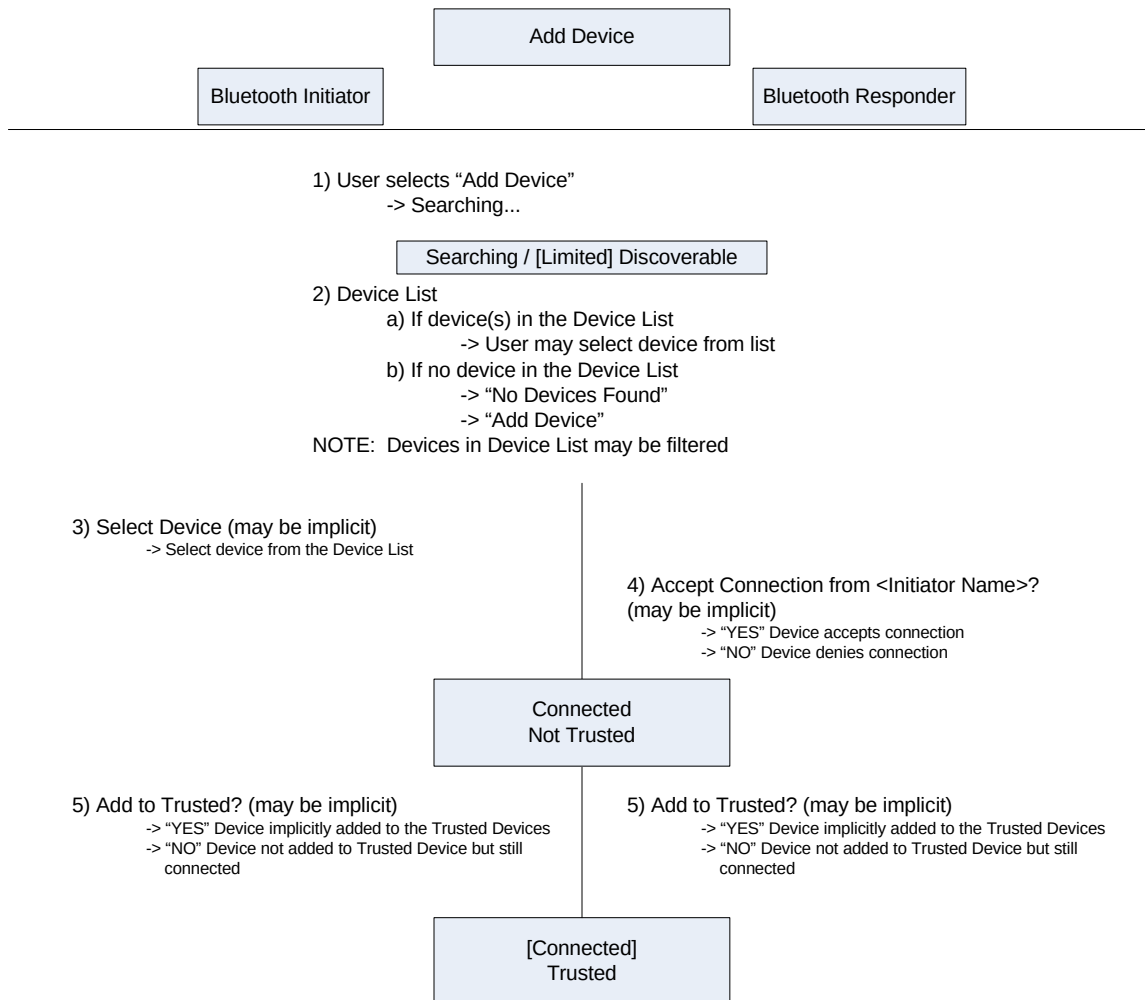


Figure 4: "Just Works" Task Flow

In the case of the “Just Works” flow, trust may be implicitly provided to the device. In the case of devices with minimal input/output capability (such as mice and headsets), trust for the primary service may be implicit.



8.2.2 NUMERIC COMPARISON TASK FLOW

The “Numeric Comparison” task flow is the *Bluetooth* Secure Simple Pairing mechanism that uses the Numeric Comparison association model. Both devices have a User Interface capable of displaying a Numeric Passkey. Bluetooth Initiator and Bluetooth Responder are in Searching / Discoverable state until roles are defined.

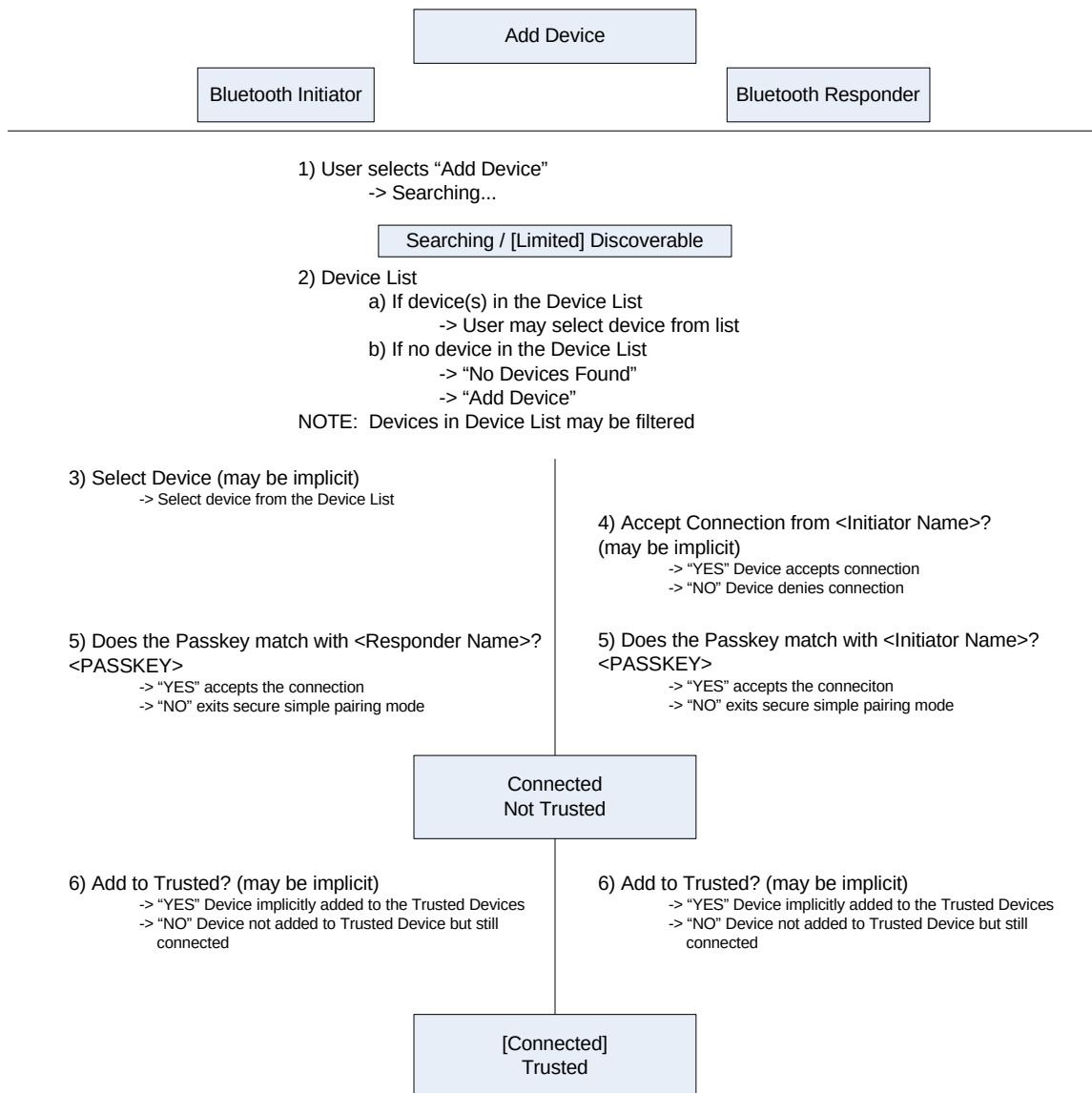


Figure 5: Numeric Comparison Task Flow

In the case of the Numeric Comparison task flow, the implementer may optionally wish to provide “device level” trust for all services available on the device as the implementer can be reasonably certain that the connection is free of security attacks. Providing device level trust should request explicit user confirmation unless explicitly provided from the context when the initial connection is made.



8.2.3 PASSKEY ENTRY TASK FLOW

The "Passkey Entry" task flow is the *Bluetooth* Secure Simple Pairing mechanism that requires entering a number on one of the devices. One of the devices must have a User Interface capable of entering a six digit number and the other device has a User Interface capable of either entering or displaying a six digit number. In the Legacy Pairing task flow one or both devices must enter a passkey to pair successfully. Bluetooth Initiator and Bluetooth Responder are in Searching / Discoverable state until roles are defined. Legacy Pairing is used when one or both devices do not support Bluetooth Secure Simple Pairing. Legacy Pairing uses a Bluetooth passkey to establish pairing between devices. Multiple permutations exist for utilizing Bluetooth passkeys between legacy devices because legacy devices may have variable passkeys, fixed passkeys or no passkey. The following task flows detail Passkey Entry and Legacy Pairing examples.

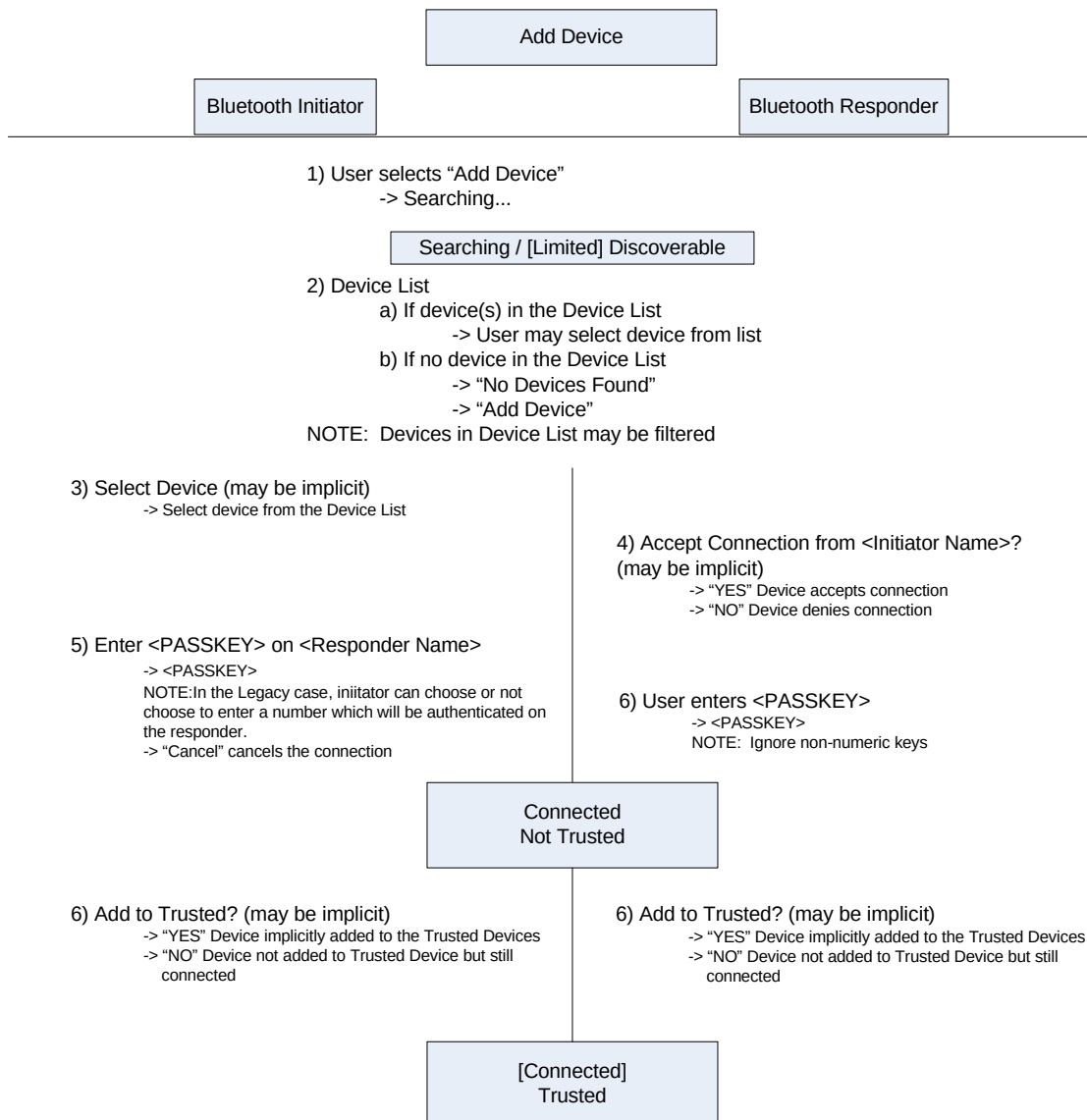


Figure 6: Passkey Entry Task Flow

8.2.4 LEGACY PAIRING TASK FLOW

When connecting to legacy devices (i.e. those devices supporting the *Bluetooth* v2.0 + EDR and prior specifications), the legacy pairing methods must be employed. It is suggested that any legacy pairing user experience should be made as similar to the *Bluetooth* v2.1 + EDR Secure Simple Pairing process.



When using SSP, the PIN may be pre-fetched prior to making a connection to the device, in order to address timeouts associated with some of the security modes found on legacy devices.

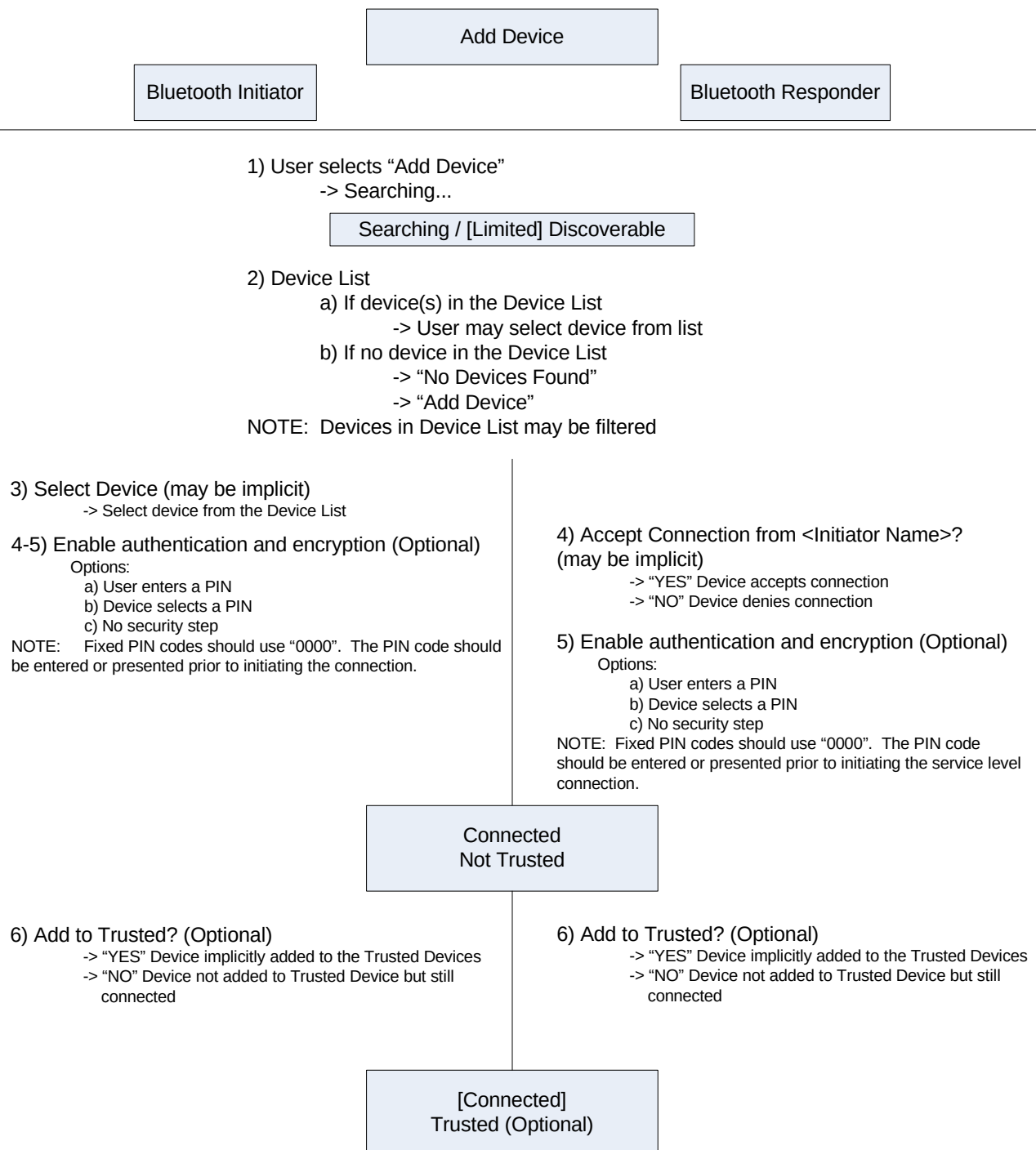


Figure 7: Legacy Pairing Task Flow⁵

⁵ In Legacy devices, the user interface concept of trust and authentication can be described by the type of passkey entry. For example, two legacy devices requiring independent passkey entry can establish a verified, trusted relationship. However, two legacy devices using a variable and fixed passkey entry may choose to establish a trusted but not verified relationship dependent on the implementation.



In the case of the Legacy Pairing task flow, the implementer may optionally wish to provide “device level” trust for all services available on the device if a random PIN of sufficient length was used as the implementer can be reasonably certain that the connection is free of Security attacks. Providing device level trust should request explicit user confirmation unless explicitly provided from the context when the initial connection is made. If a low entropy passkey (e.g. ‘0000’) was used, then the implementer should be cautious not to provide device level trust unless suitable for the use-case in question.

8.3 ADD SERVICE

8.3.1 ADD SERVICE TASK FLOW

This flow is used when a user wishes to increase the strength of the security attack protection for a Service that may require higher security. This flow is primarily used when the initial Secure Simple Pairing was “Just Works” (with no security attack protection) and the user desires to use a different Service (e.g. to exchange sensitive information such as credit card numbers, medical information, etc.) via a Not Trusted⁶ Service⁷. An example is when pairing is preformed in an isolated environment such as a secure facility or screen room. The Secure Simple Pairing algorithms do not provide any inherent security attack protections but “Just Works” could provide an acceptable level of security if other protections are included.

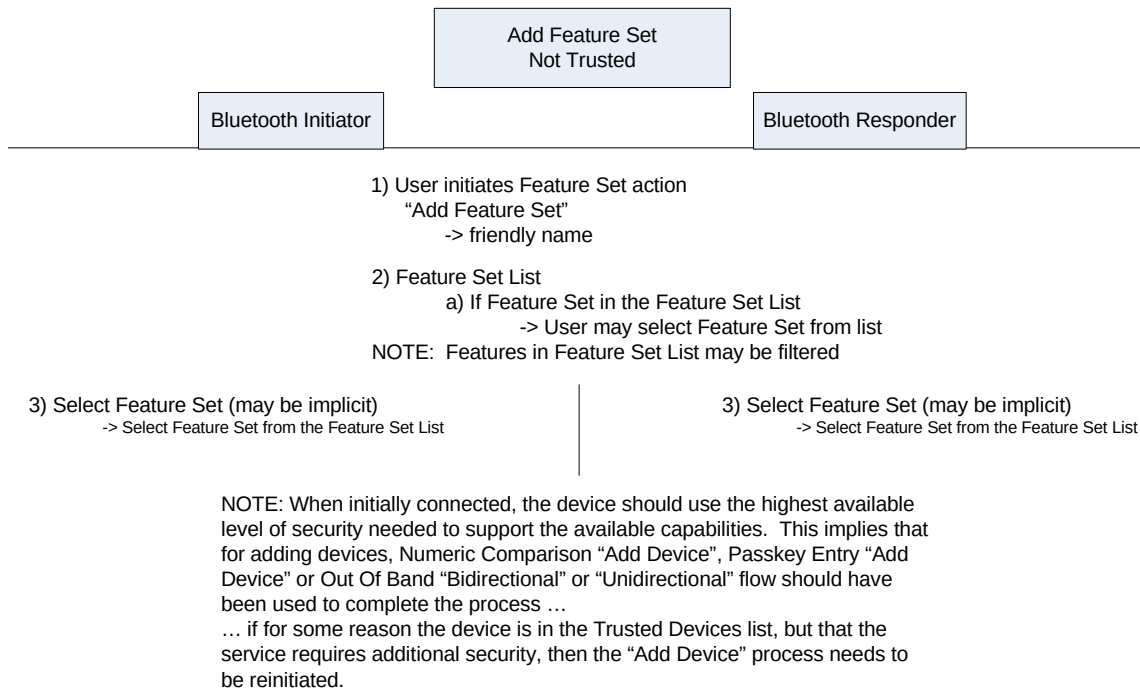


Figure 8: Add Service Task Flow

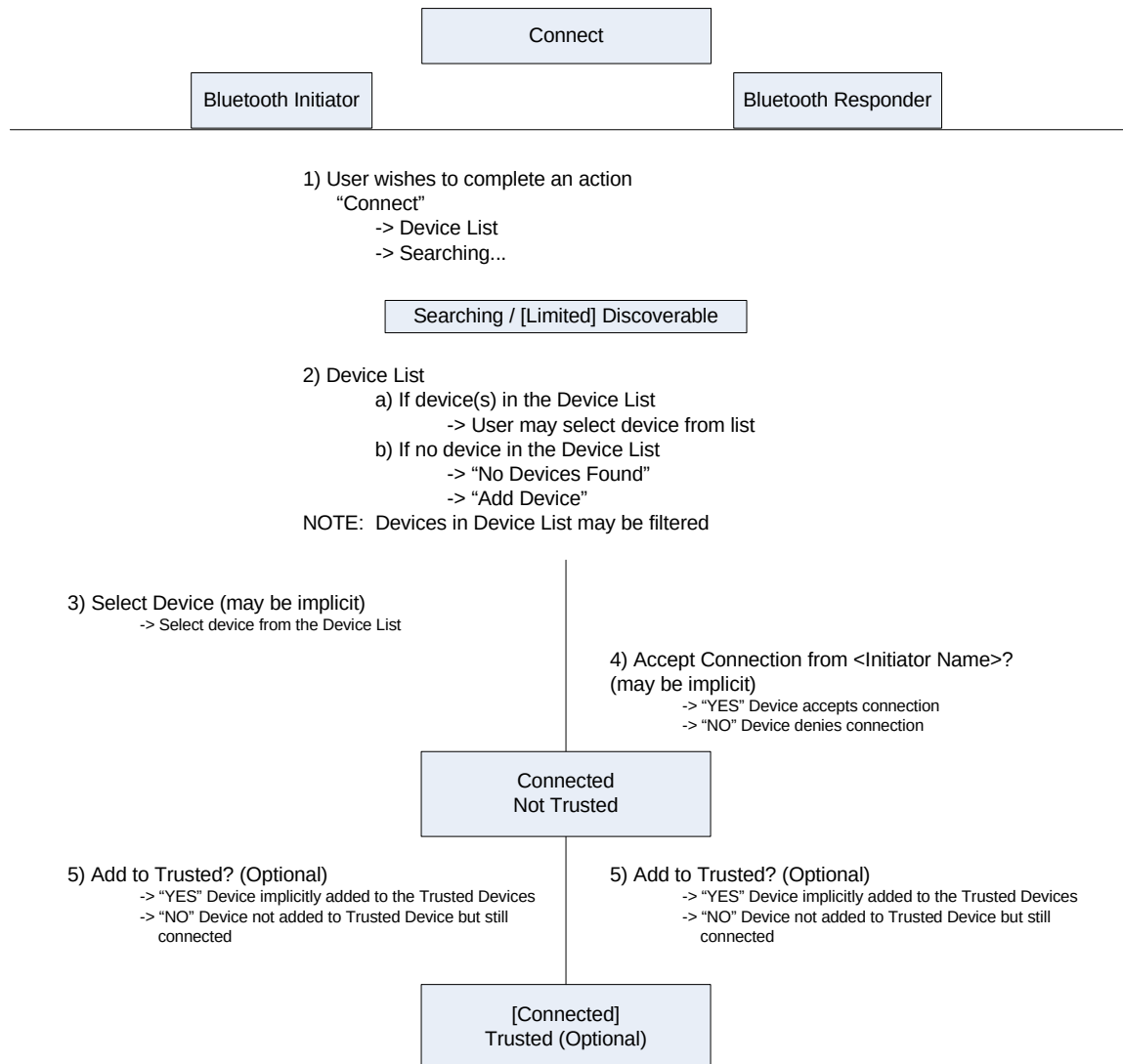
⁶ Not Trusted – See definition in reference [1].

⁷ Feature [Set] - See definition in reference [1].



8.4 CONNECT TASK FLOW

Used when connecting to a device in the Device List (Trusted Device(s) or Discovered Device(s)) as referenced in the Bluetooth Secure Simple Pairing Lexicon [1]. Bluetooth Initiator and Bluetooth Responder are in Searching / Discoverable state.



|Figure 9: Connect Task Flow

When “connecting” from one device to another, the connection is generally the result of a specific action that the user wishes to perform. As such, the step to add the device to the users Trusted List is optional, especially if the user’s goal is simply to complete the action in question. In cases where the user wishes to establish a more permanent connection, the step may be implicit or required (i.e. connecting a HID device).

8.5 SEND / RECEIVE TASK FLOW

Used when sending or receiving to a device in the Device List (Trusted Device(s) or Discovered Device(s)) as referenced in Bluetooth Secure Simple Pairing Lexicon [1]. Known Devices may not require an additional Device Name request. Bluetooth Initiator and Bluetooth Responder are in Searching / Discoverable state.



When "Send" is selected the device takes on the role of Initiator in the Searching state. When "Receive" is selected the device takes on the role of Responder in the Discoverable state.

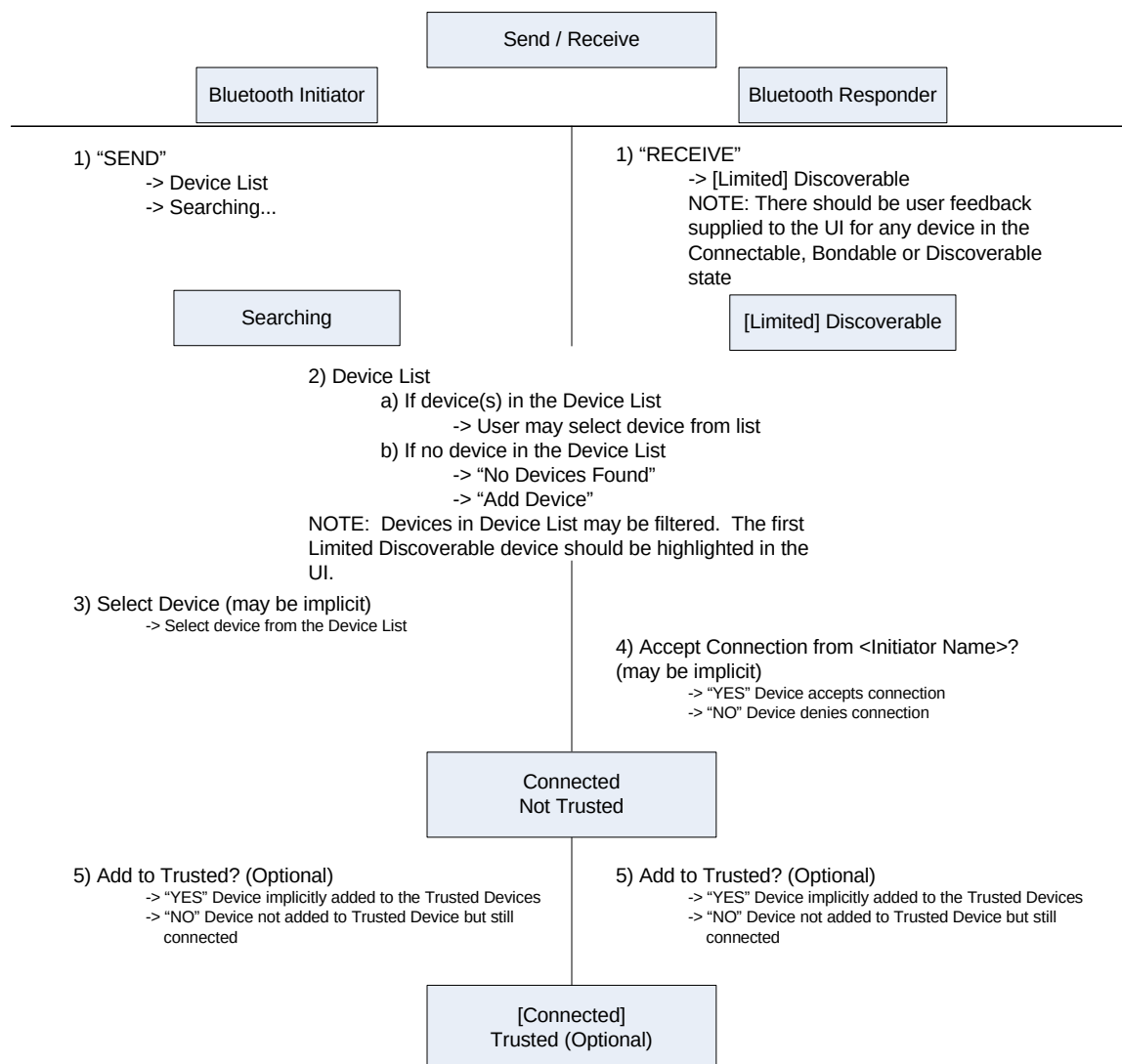


Figure 10: Send / Receive Task Flow

When sending and receiving data from one device to another, the step to add the device to the user's Trusted List is optional, especially if the user's goal is simply to complete the action in question. Once the send/receive action is complete, if the user wishes to establish a more permanent connection, user may be given the option to trust the device and thus allow it to reconnect without requiring explicit user acceptance for the connection.



8.6 REJECT / BLOCK CONNECTION FLOW

This flow is performed when the user decides to reject an incoming connection from a remote device. The Device will then not appear in the Trusted Device List. To connect to the device in the future after blocking the device, the user must either select the device in the Blocked Device List and connect to it directly or actively remove the blocked device from the Blocked Device List to re-enable the device to be displayed in the Device List when searching.

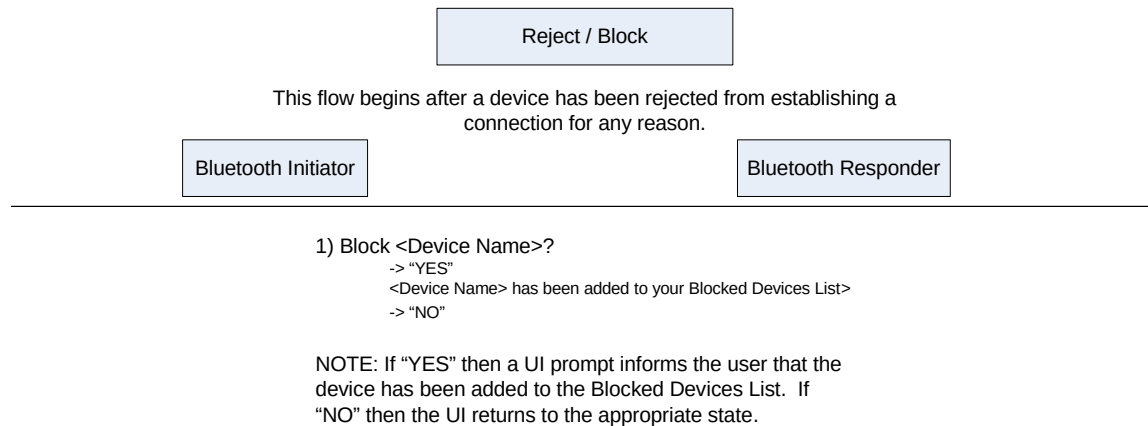


Figure 11: Reject Connection Task Flow

8.7 OUT OF BAND TASK FLOW

The "Out Of Band" (OOB) task flow is the *Bluetooth* Secure Simple Pairing mechanism that requires a secondary secure non-*Bluetooth* channel to pass information when pairing two devices. The secondary secure non-*Bluetooth* channel must be able to exchange OOB service discovery information and may be established between the two devices prior to initiating the pairing process. The OOB Initiator and OOB Responder are in a dormant state until the roles are defined by the specific OOB information exchanged. The Searching / Discoverable state may be entered at any point relative to the exchange of OOB information; however, in each case the OOB information must be passed between the two devices prior to initiating the *Bluetooth* Secure Simple Pairing process although a non-secure *Bluetooth* wireless connection may already exist between the two devices in question.

There are two broad categories of OOB task flows:

- 1) Activating the OOB mechanism and subsequently initiating the *Bluetooth* pairing process.
- 2) Initiating the *Bluetooth* connection (no pairing) and subsequently exchanging OOB information.

There are two broad OOB implementations supported by *Bluetooth* Secure Simple Pairing:

- 1) Bidirectional: This implementation is based on the principle that two devices may be connected to each other via bidirectional OOB channel and transfer information in real-time. Example technologies that support the Bidirectional implementation are cabled connections, infrared, and Near Field Communication (NFC) Devices.
- 2) Unidirectional: This is a catch-all for all other implementations where *Bluetooth* Secure Simple Pairing information may be passed via mechanisms that only pass information in one direction. Examples include NFC tags, secure email attachments, encrypted Short Message Service (SMS), and placing encrypted files on a server which can be read by a device. Other implementations may also include memory sticks and barcode readers/printers.

In this document we describe a task flow for each of the two mechanisms. Each of the two broad categories for OOB pairing may be applied to any of the two supported *Bluetooth* Secure Simple Pairing implementations (Bidirectional, Unidirectional). This implies that a user may either initiate the *Bluetooth* Secure Simple Pairing process via one of the two *Bluetooth* Secure Simple Pairing implementations OR the user may initiate a *Bluetooth* connection, and then apply one of the two *Bluetooth* Secure Simple Pairing implementations. The



searching / discoverable state can cease once the connection information has been passed via the OOB channel. This can shorten the time for a device to be in the searching / discoverable state.

8.7.1 BIDIRECTIONAL TASK FLOW

The Bidirectional task flow is used when two devices can communicate via an OOB mechanism and start *Bluetooth* Secure Simple Pairing process in real-time by passing data in both directions between the *Bluetooth* Initiator and the *Bluetooth* Responder. As an example for NFC devices, the two devices are physically brought together and the *Bluetooth* Secure Simple Pairing information is passed via the NFC channel thus initiating the *Bluetooth* Secure Simple Pairing process. The task flow may also be used with two devices equipped with a complementary mechanism for physically passing the *Bluetooth* Secure Simple Pairing information via a cable or other digital file transfer media. NOTE: Due to the fact that the user is actively plugging a cable into both devices, the confirmation step may be implicit.

In the Bidirectional task flow, the OOB Initiator is also the *Bluetooth* Initiator. This is because the OOB Initiator receives a response containing the pertinent OOB information from the OOB Responder and thus can effectively initiate the *Bluetooth* Secure Simple Pairing process.

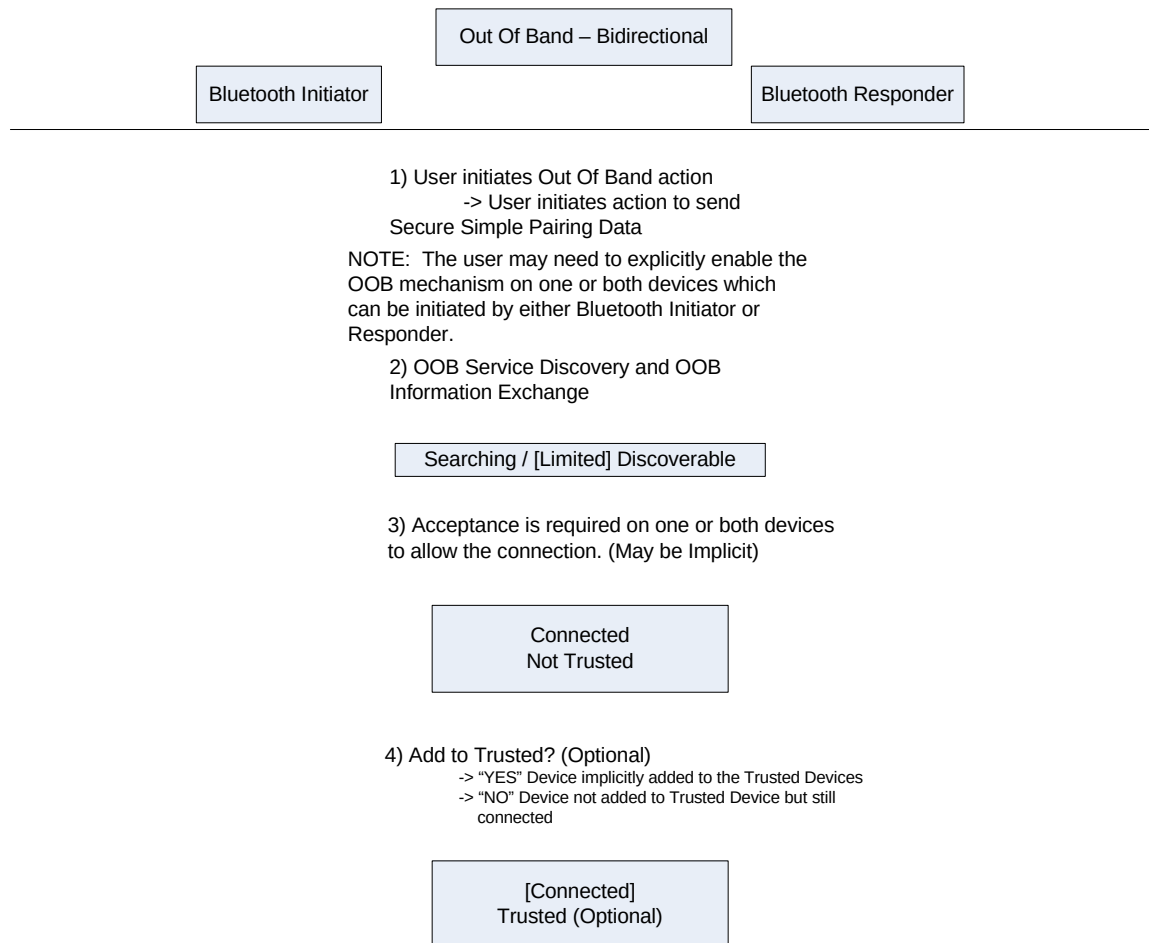


Figure 12: OOB Bidirectional Task Flow



8.7.2 UNIDIRECTIONAL TASK FLOW

The Unidirectional task flow is used to capture all scenarios where two devices are equipped with the same mechanism for transferring *Bluetooth* Secure Simple Pairing information via an OOB channel but where the information is passed only in one direction. Due to the complexity of this process, implementers are encouraged to walk through all of the possible permutations that apply to their applications before implementing this form of OOB communication process.

For some unidirectional task flows, the OOB initiator sends the *Bluetooth* Secure Simple Pairing information to the OOB responder (i.e. via an email, file, token, etc.). Since this information is not explicitly acknowledged by the OOB responder, the OOB initiator then becomes the *Bluetooth* Responder as the OOB responder initiates the *Bluetooth* Secure Simple Pairing process. Applications can be used to handle the OOB information. Such examples are an SMS or email with the configuration information listed that the device can use.

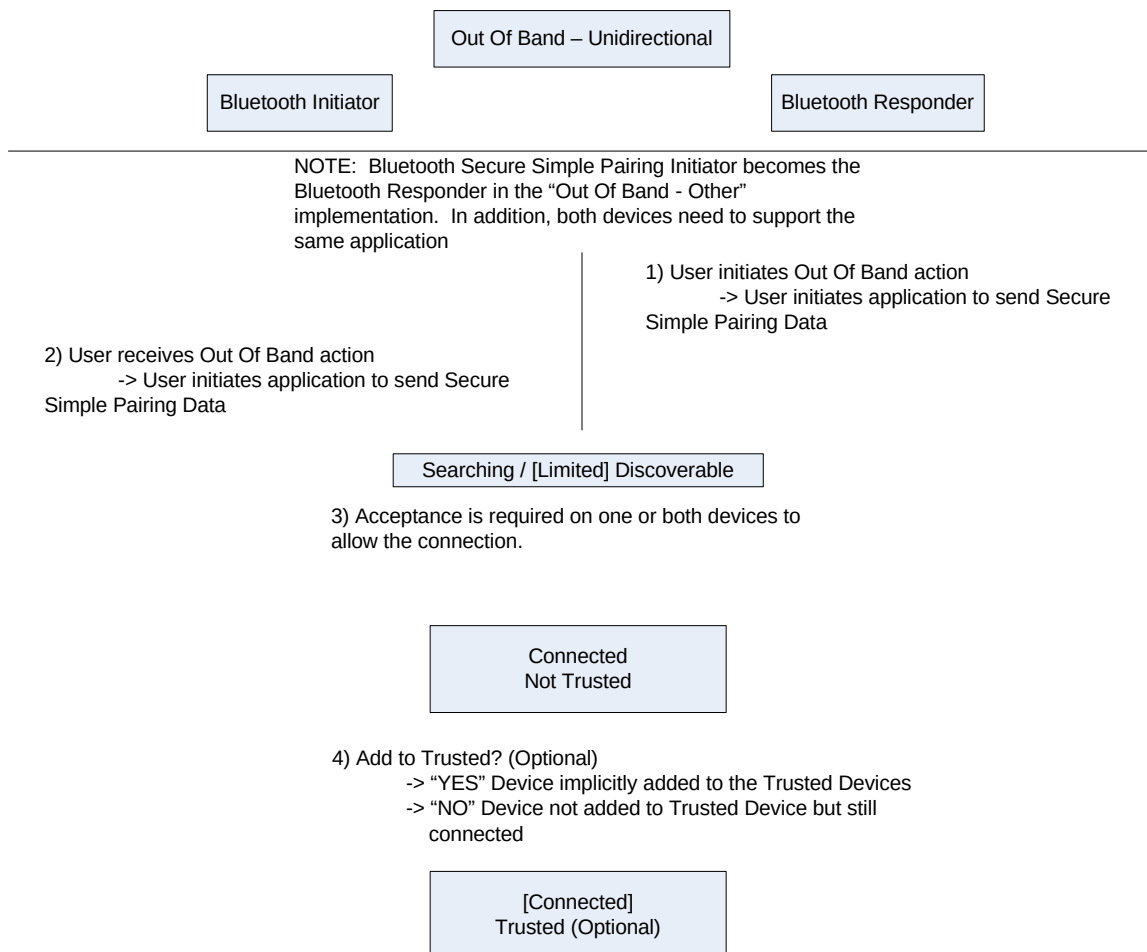


Figure 13: OOB Unidirectional Task Flow

In both the OOB Bidirectional and OOB Unidirectional task flows, the final step to trust the device may be either implicit (e.g. for single service devices such as a mouse), required, or omitted entirely depending on the device and the context in which the connection is made.

8.7.3 ADD DEVICE TASK FLOW

The OOB add device task flow is used when a user initiates an add device task flow for starting the *Bluetooth* Secure Simple Pairing *Bluetooth* process. The OOB mechanism is utilized after determining the I/O capabilities of the two devices.

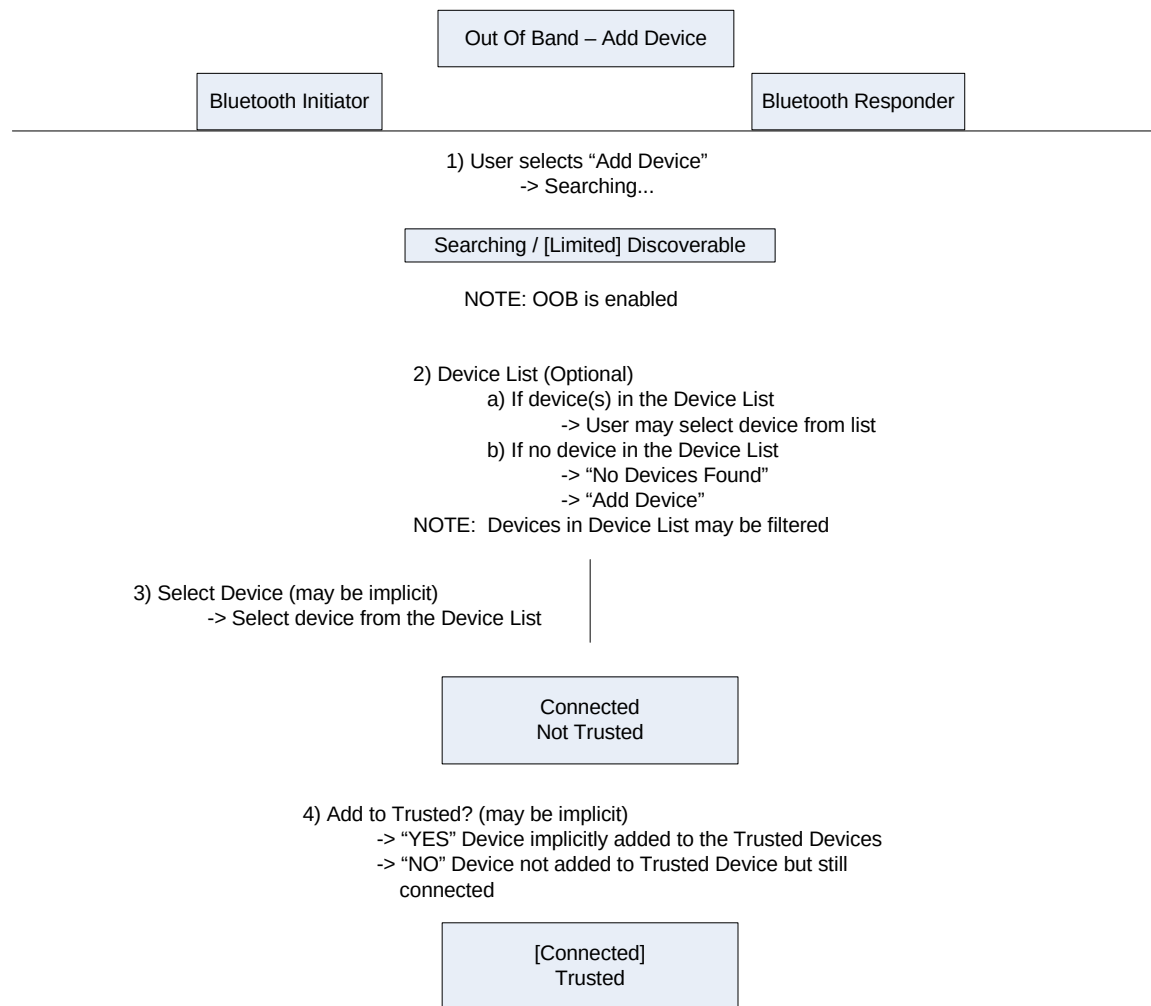


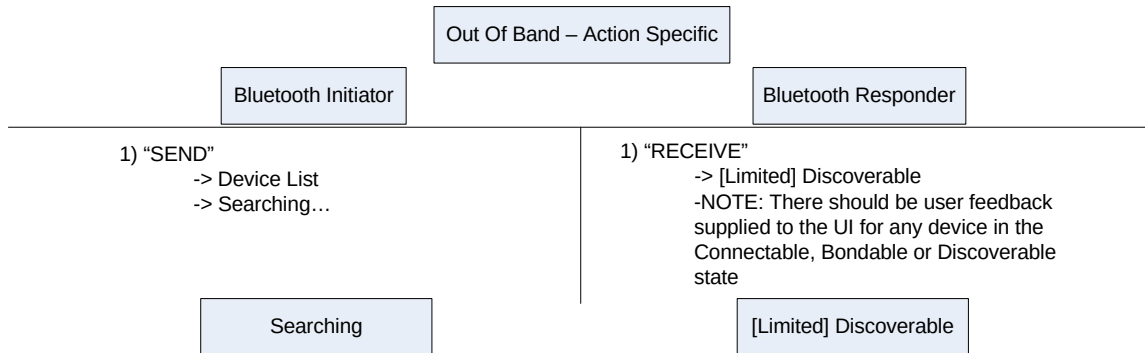
Figure 14: OOB "Add Device" Task Flow

When adding a device, the assumed intent is to add the device to your Trusted Device list so as to use the services on the device. If an OOB technique is used when adding a device, then the connection is assumed to be free from Security attack and thus device level trust may be implicitly provided for all profiles available on the device if warranted by the use case and context.



8.7.4 ACTION SPECIFIC TASK FLOW

The OOB action specific task flow is used when a user performs an action that requires a *Bluetooth* Secure Simple Pairing between *Bluetooth* devices.



NOTE: At this point, the OOB channel is enabled. See "OOB – Unidirectional" or "OOB – Bidirectional" flows for the rest of the process.

Send / Receive is shown as a generic action specific task. Other action specific tasks are possible. Send / Receive could be initiated by either side or be bidirectional depending on the application. In most cases, the Sender is the Initiator and the Receiver is the Responder.

Figure 15: OOB "Action Specific" Task Flow



9 User Interface Flows⁸

9.1 FACTORY DEFAULT USER INTERFACE FLOW

The User Interface associated with the Factory Default action is not covered in the scope of this document.

9.2 ADD DEVICE

9.2.1 JUST WORKS UI FLOW

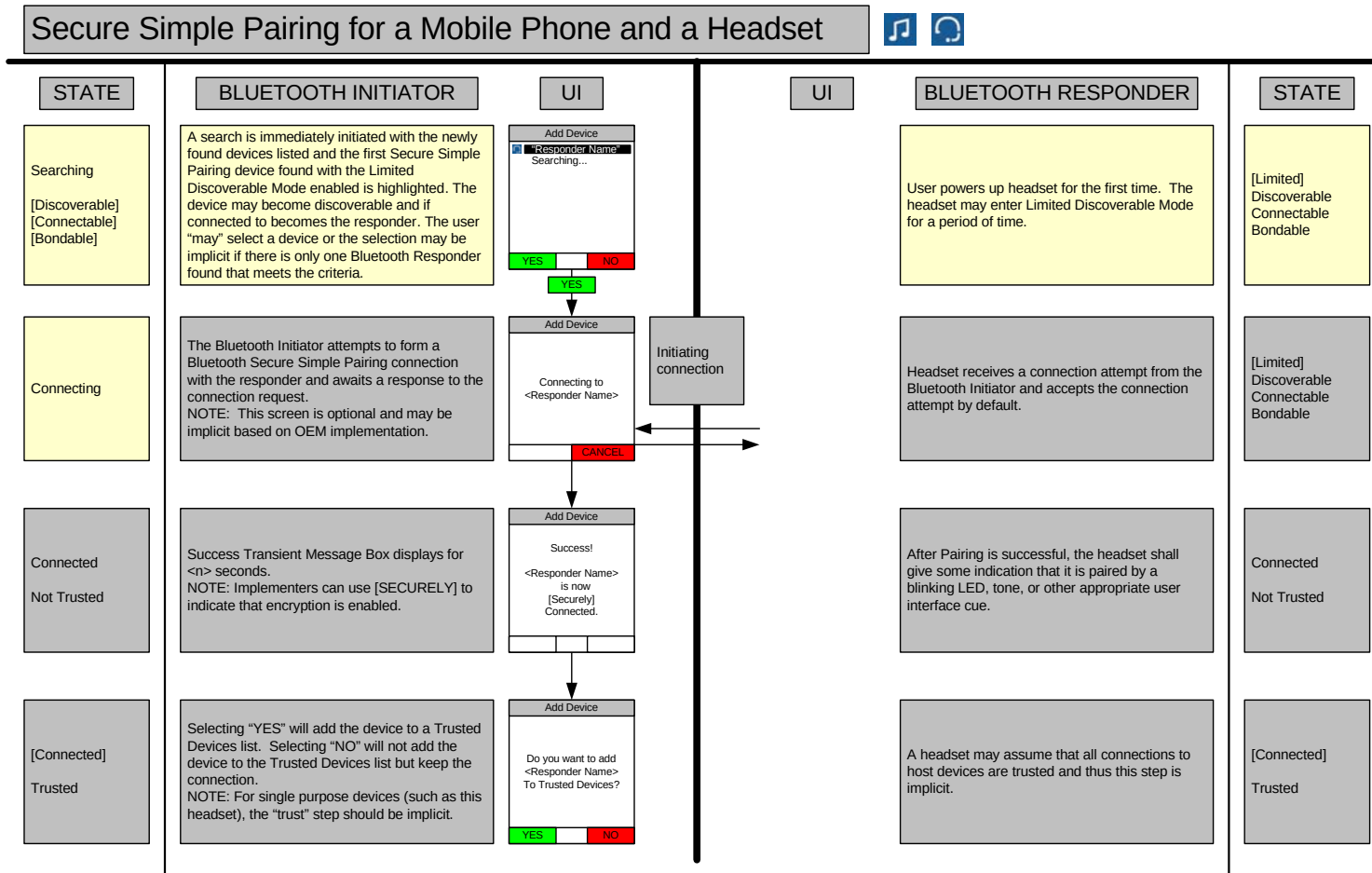
The “Just Works” user interface flow is used when one or more devices do not have a User Interface capable of displaying or entering a six digit number (*Simple I/O*) or passing information via an OOB channel. It is suggested that user action to “Add Device” may have an easily accessible UI mechanism to access this flow.

⁸ Refer to Section 4, Figure 2 for transition states.



9.2.1.1 MOBILE PHONE TO HEADSET

The following is an example for a mobile phone and headset for Just Works scenario. The implementers may decide to show already connected devices in their UI.



9

Figure 16: "Just Works" Headset Pairing User Interface Flow

⁹ The visible state is where a device can be found when searching but not connected to. Devices should be discouraged to using this visible state unless it is used in conjunction with a connectable state.



9.2.1.3 V-CARD TRANSFER

The standard V-Card transfer flow does not take into account some of the optimizations that are possible when envelope information is available from the v-card or other transfer prior to sending the full file. The more generic approach, while not as elegant, can be used for all types of transfers whether or not envelope information can be provided immediately upon establishing the service level connection.¹⁰

¹⁰ In the “Just Works - Standard” case user confirmation is triggered when the RFCOMM connection is established



V-Card Transfer - Just Works Standard

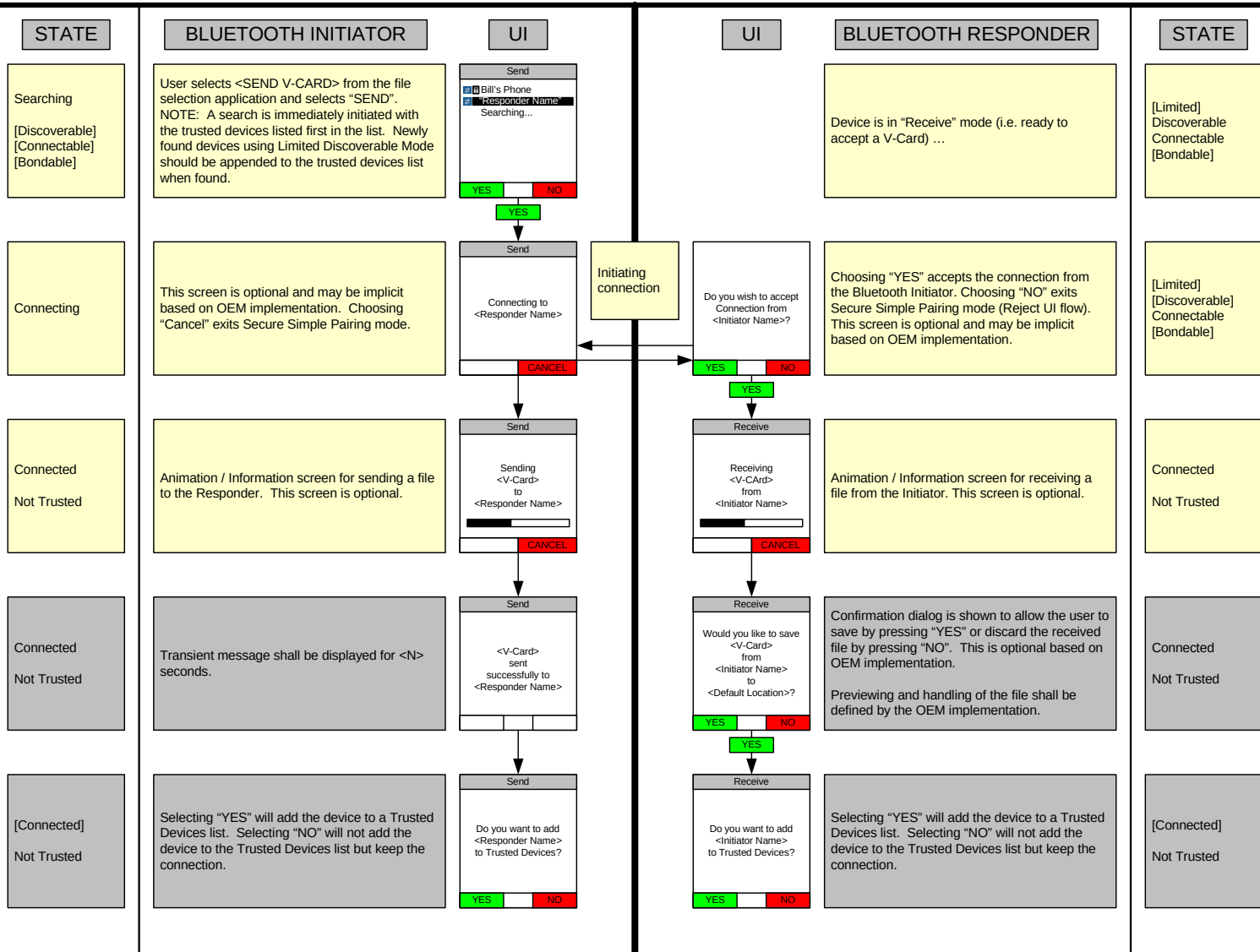


Figure 18: V-Card Transfer – Standard "Just Works" UI Flow



9.2.1.4 V-CARD TRANSFER –OPTIMIZED

This V-Card transfer flow has been optimized to take into account the envelope information that can be transferred initially to help the receiving side better message the contents of the information transfer to the user.¹¹

¹¹ In the “Just Works - Optimized” case user confirmation is triggered when the OBEX connection is established



V-Card Transfer - Just Works Optimized

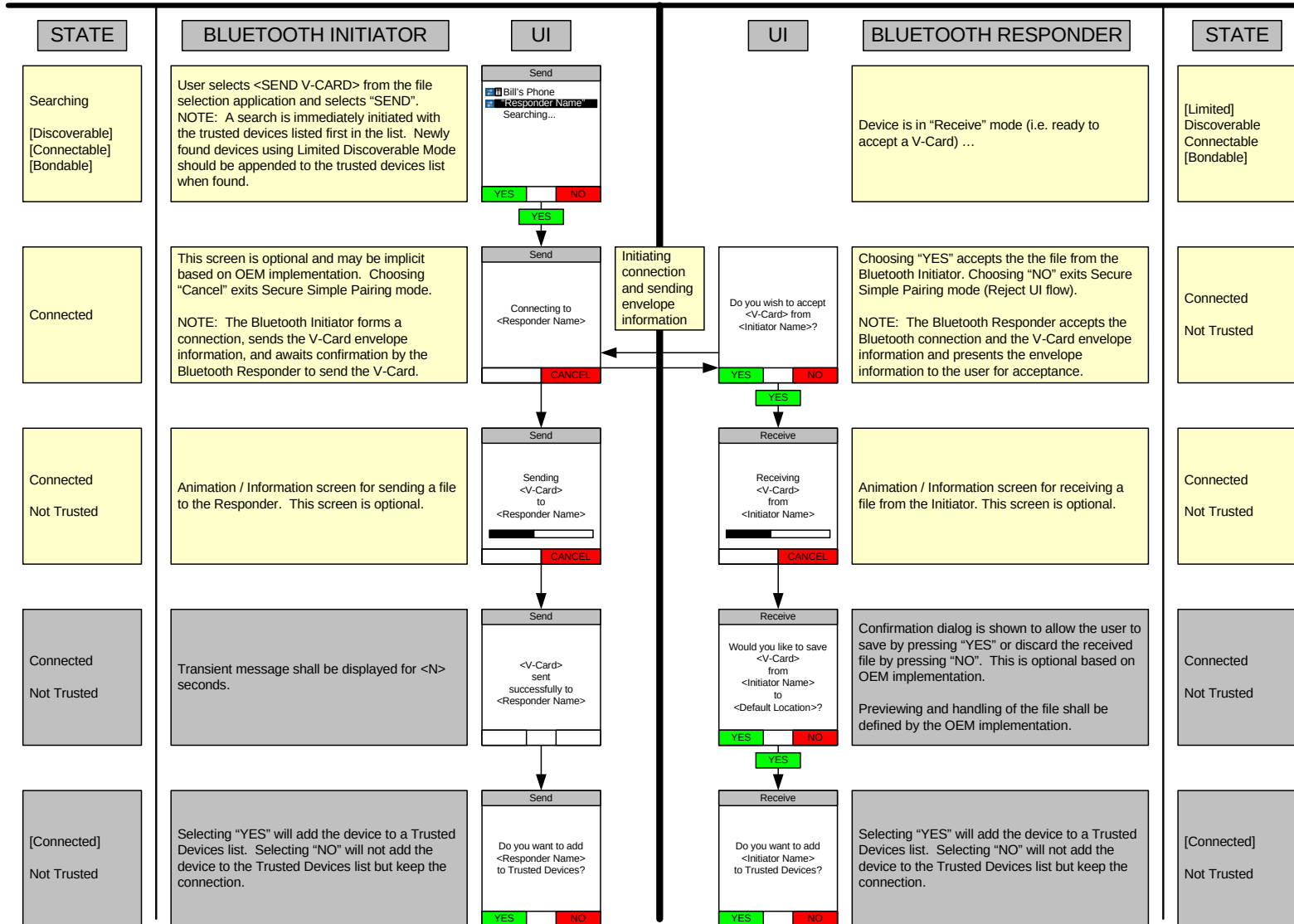


Figure 19: V-Card Transfer - Optimized "Just Works" UI Flow



9.2.2 NUMERIC COMPARISON UI FLOW

The Numeric Comparison user interface flow is used when both devices have a User Interface capable of displaying a Numeric Passkey.

9.2.2.1 PC TO MOBILE PHONE

In this example both devices are in the searching and discoverable mode but they both may not need to be in these states in order to start a connection.

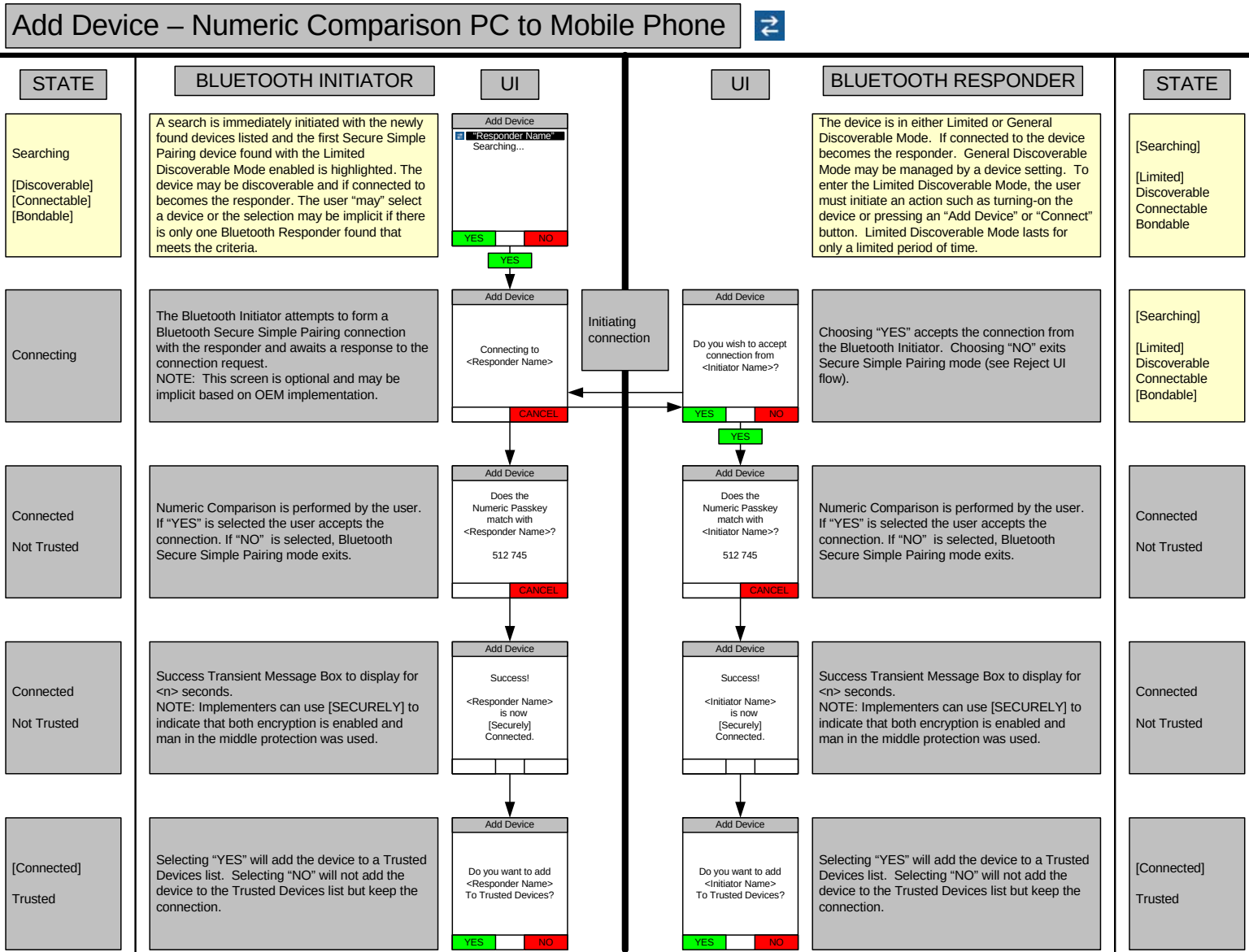


Figure 20 Add Device - Numeric Comparison User Interface Flow



9.2.2.2 SEND FILE - MOBILE PHONE TO MOBILE PHONE

When sending a file between devices, trust is not generally provided implicitly, thus a UI screen should be provided if a trust relationship is to be setup.



Secure Simple Pairing - Sending a Picture between two Mobile Phones

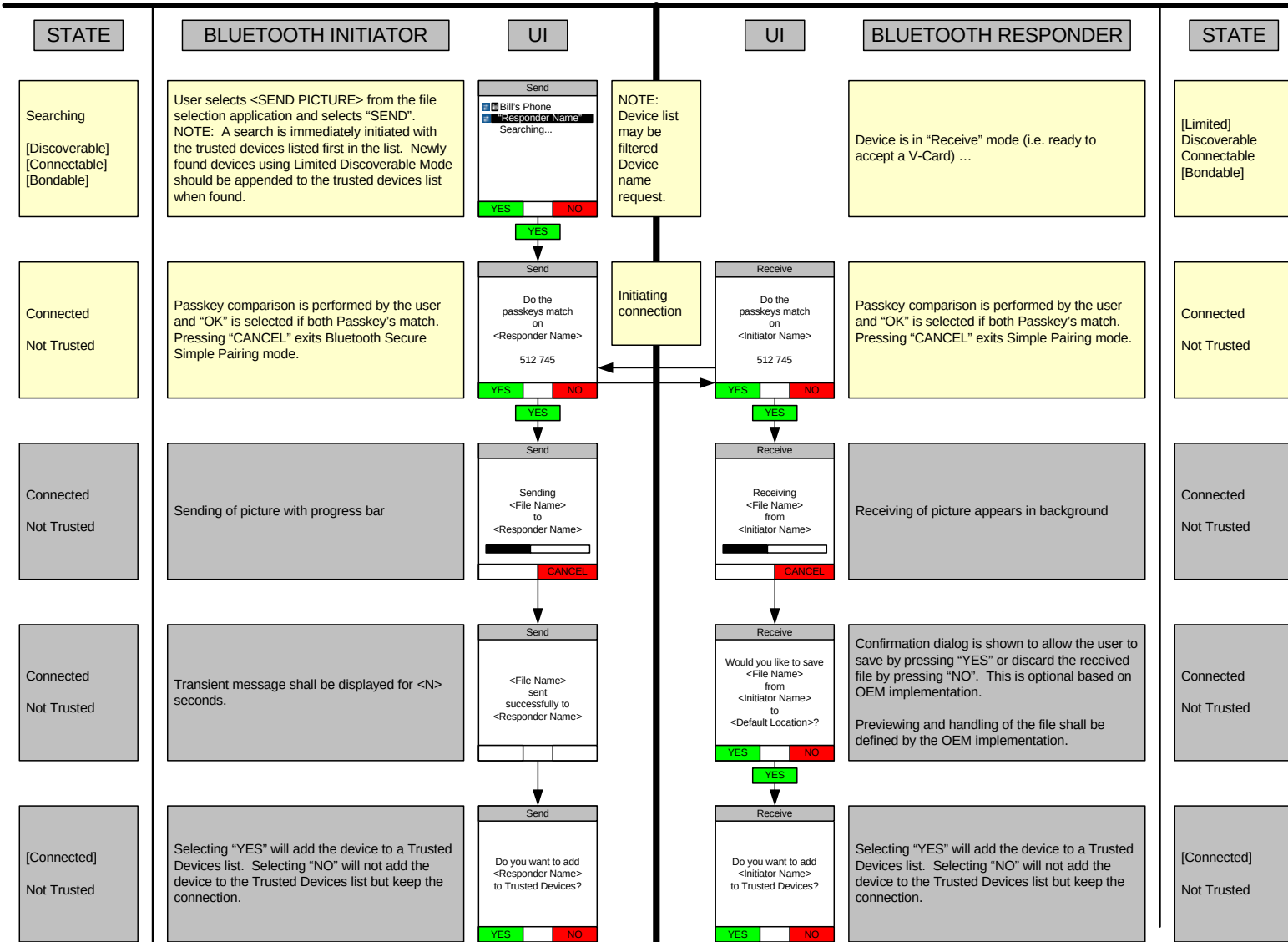


Figure 21: Numeric Comparison - Send File (Picture) User Interface Flow



9.2.3 NUMERIC [PASSKEY] ENTRY AND LEGACY PAIRING UI FLOW

The Numeric [Passkey] Entry user interface flow is used when both devices support *Bluetooth* Secure Simple Pairing with one device having a User Interface capable of entering a Numeric Passkey and another device capable of displaying a Numeric Passkey. Legacy Pairing is used when one or more devices do not support *Bluetooth* Secure Simple Pairing.



9.2.3.1 DEVICES WITH A HID KEYBOARD

Secure Simple Pairing for devices with a HID Keyboard

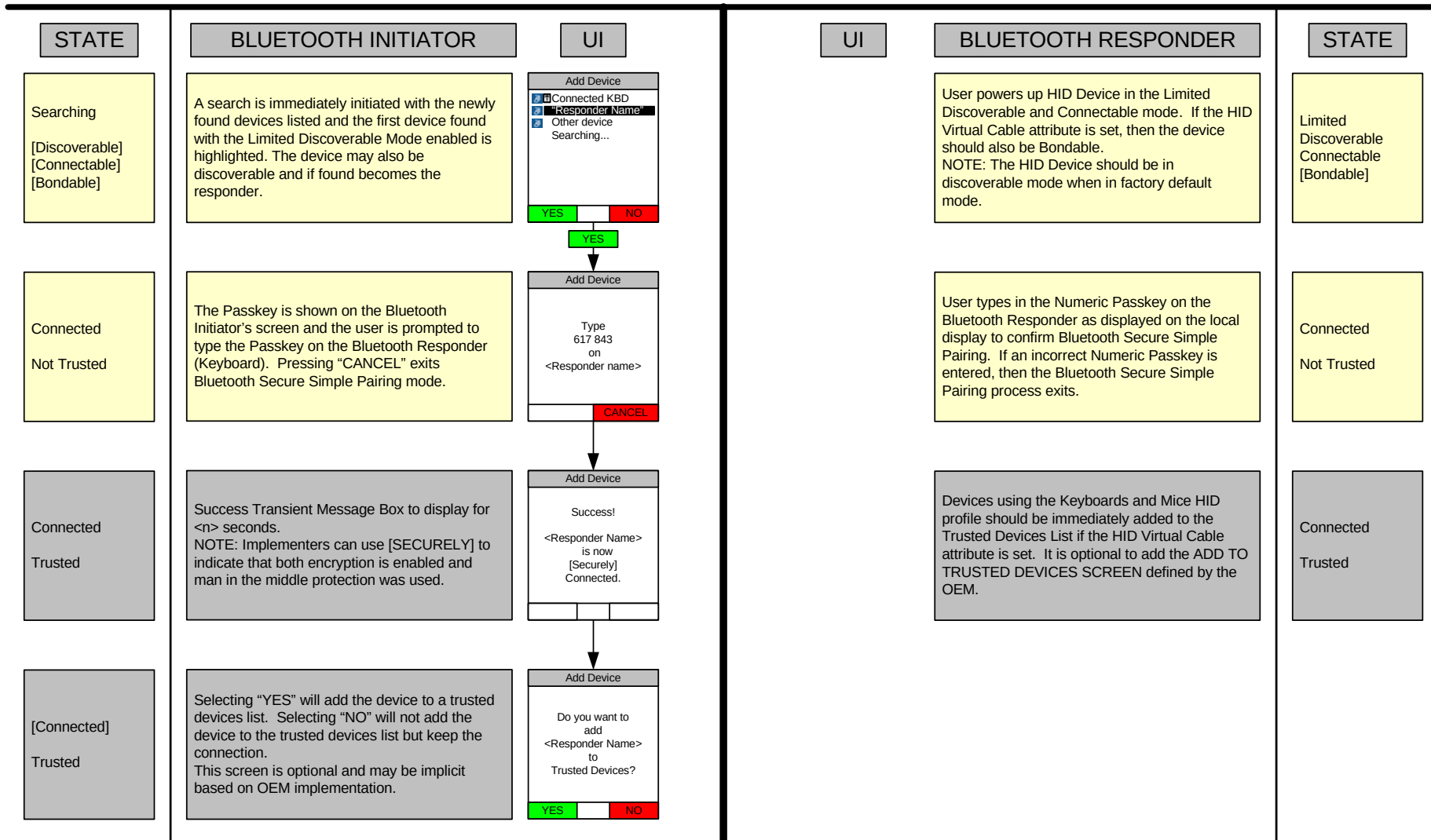


Figure 22: Secure Simple Pairing for Devices with a HID Keyboard UI Flow



9.2.3.2 LEGACY HID PASSKEY PAIRING WITH A PC

Legacy HID PIN Pairing with a PC and a Phone

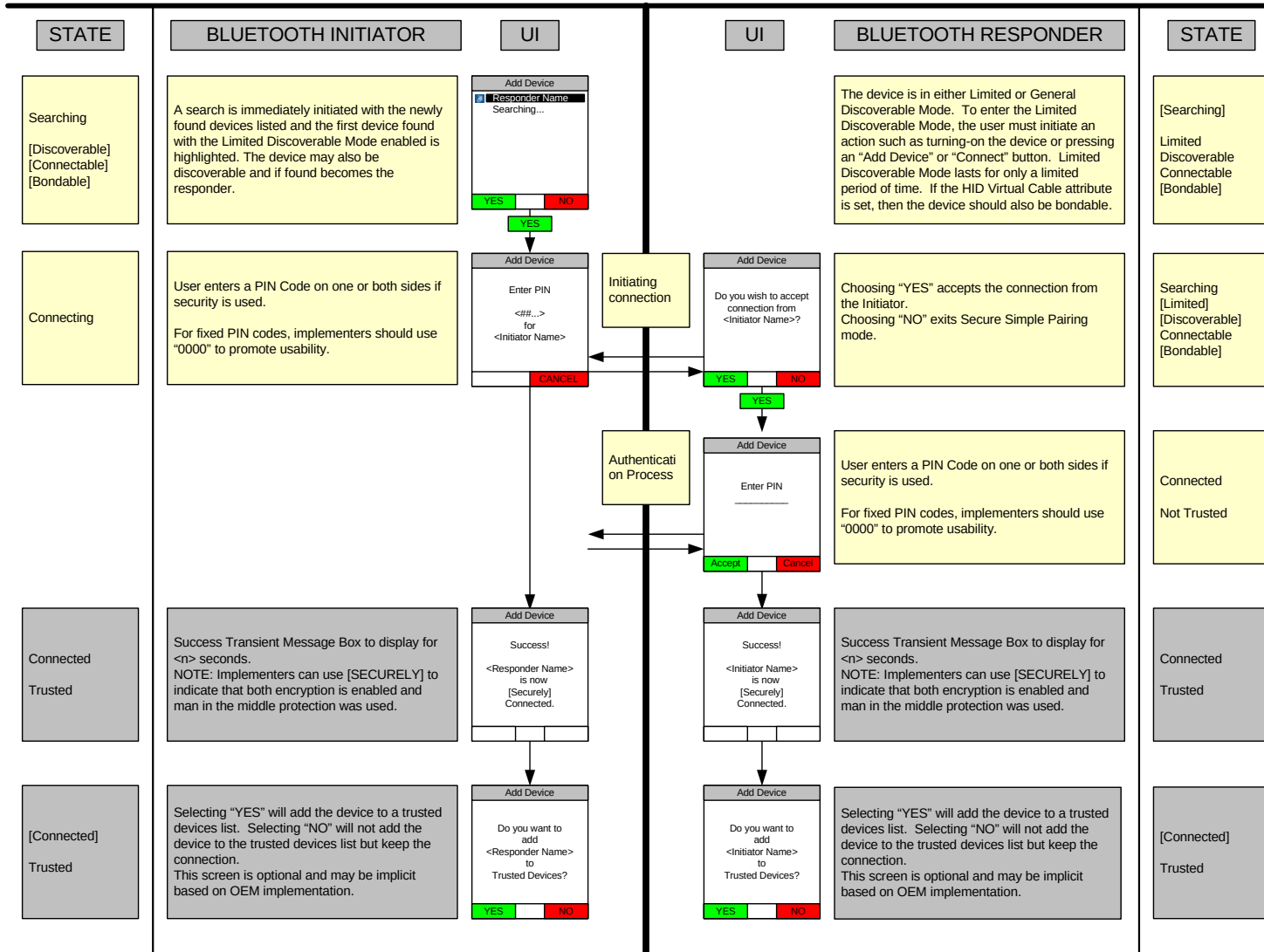


Figure 23: Legacy HID Passkey Pairing With a PC User Interface Flow



9.2.4 FIXED PIN PAIRING FOR DEVICES WITH A HEADSET

Fixed PIN Pairing for devices with a Headset

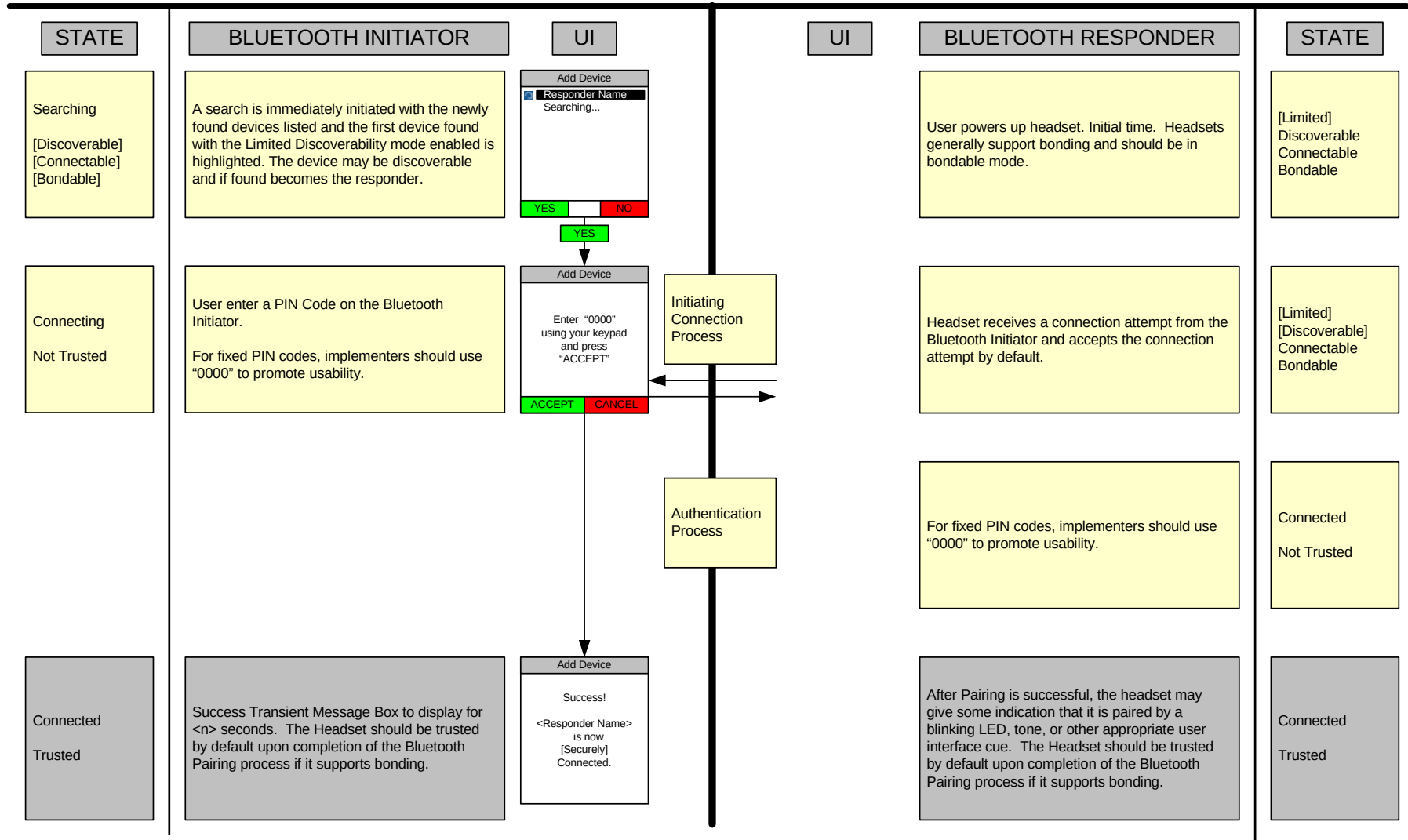


Figure 24: Fixed PIN Pairing for Devices with a Headset User Interface Flow



9.2.5 OUT OF BAND UI FLOWS

This flow is used to add a Mobile Phone to a PC using an OOB mechanism with *Bluetooth* Secure Simple Pairing.

The OOB user interface flow is used when both devices leverage a compatible, non-*Bluetooth* channel to pass the pairing information. The *Bluetooth* Secure Simple Pairing information must be shared prior to initiating the OOB pairing process. The protocol does not define a specific OOB mechanism and leaves the specifics to the implementer's discretion. The following example outlines the user interface flow after the OOB information exchange has taken place. Using OOB implicitly selects to connect to a specific device that may or may not be in the device list yet.



Add Device – “Out Of Band” with a PC and Mobile Phone

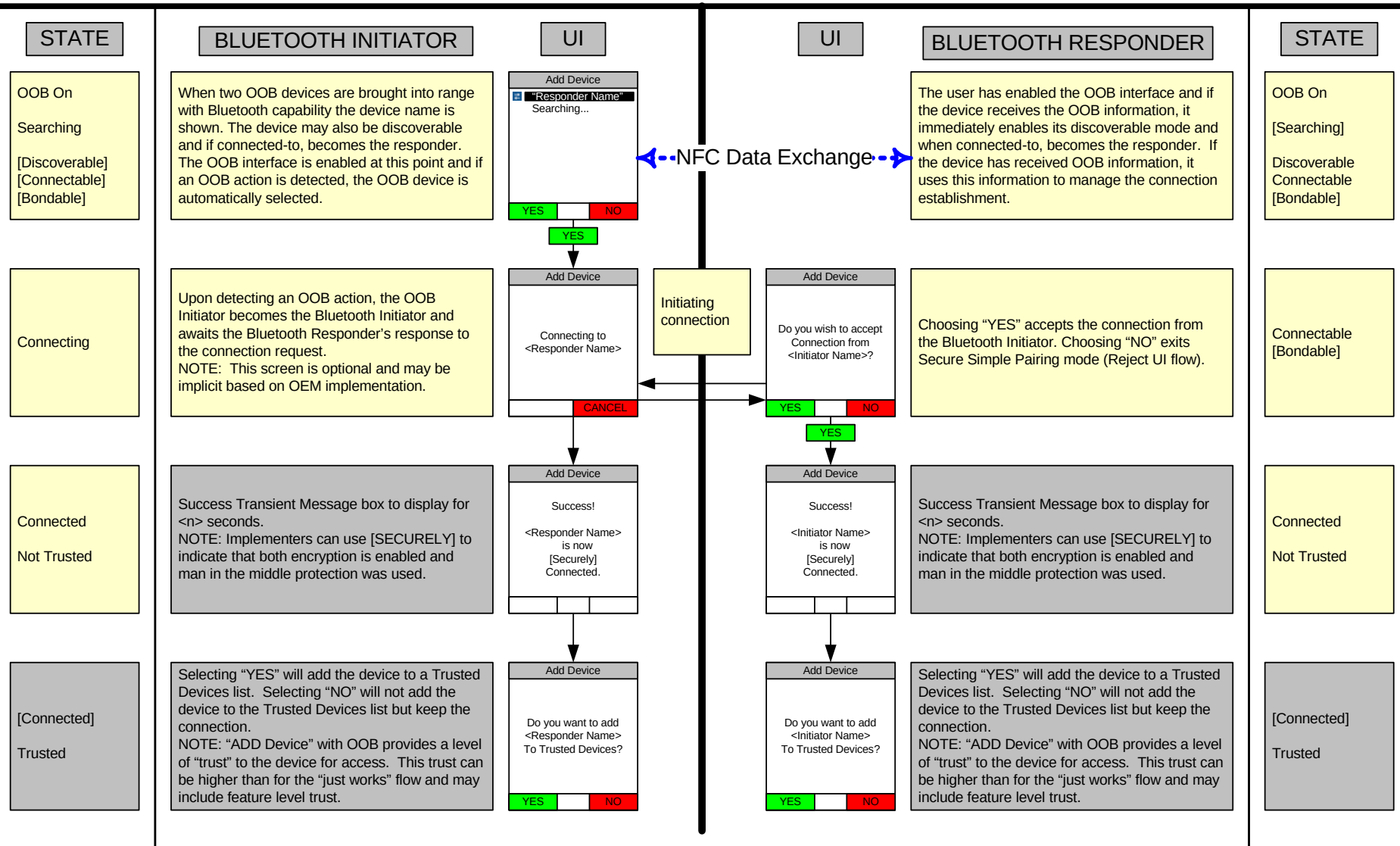


Figure 25: PC to Mobile Phone OOB User Interface Flow



9.2.5.1 ADD DEVICE – USING NFC (CAMERA TO PRINTER)

Add Device – OOB using NFC (Camera to Printer)

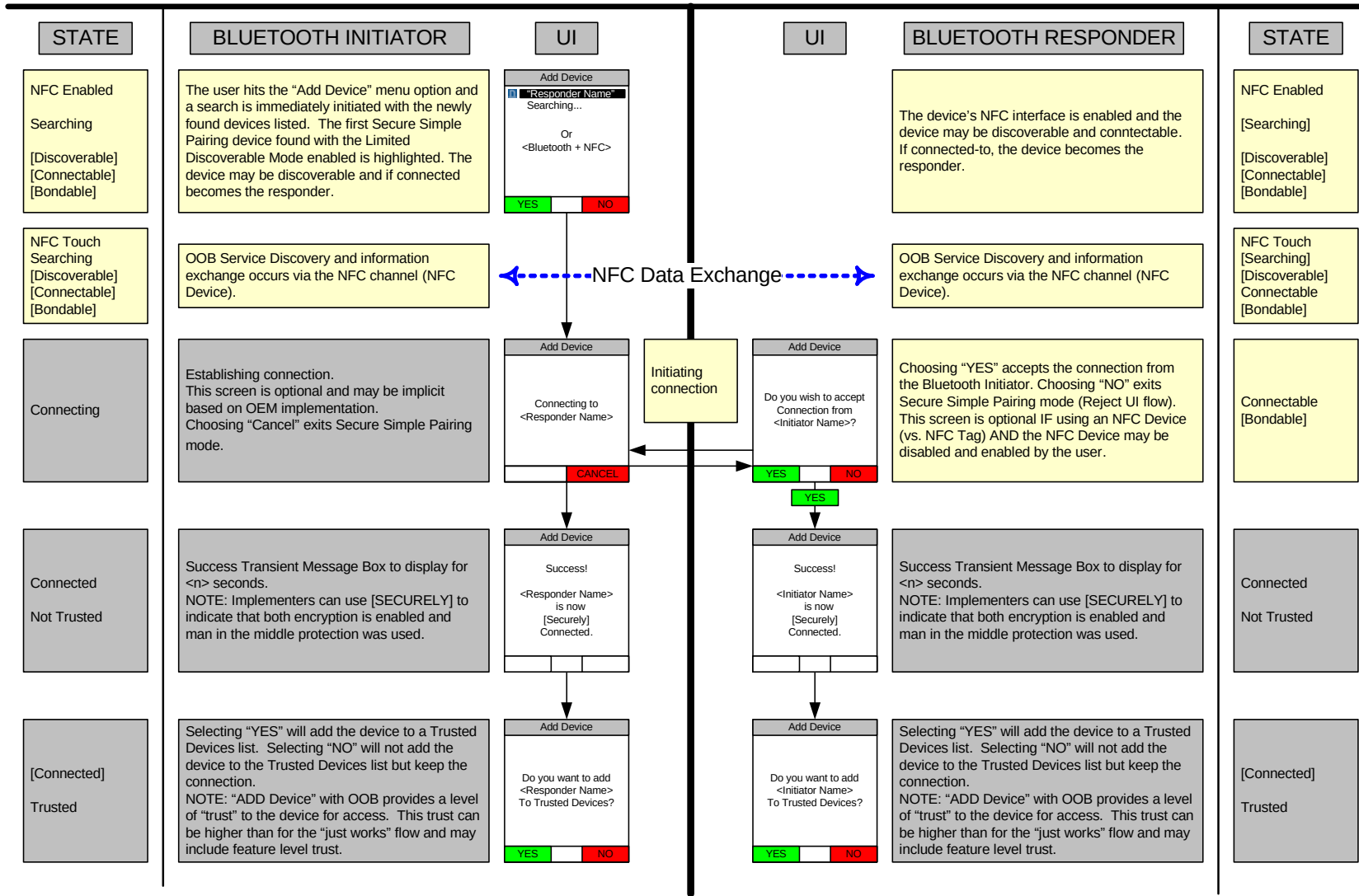


Figure 26: Camera to Printer NFC User Interface Flow



9.2.5.2 UNIDIRECTIONAL PDA TO PDA

"OUT OF BAND" - Unidirectional PDA to PDA

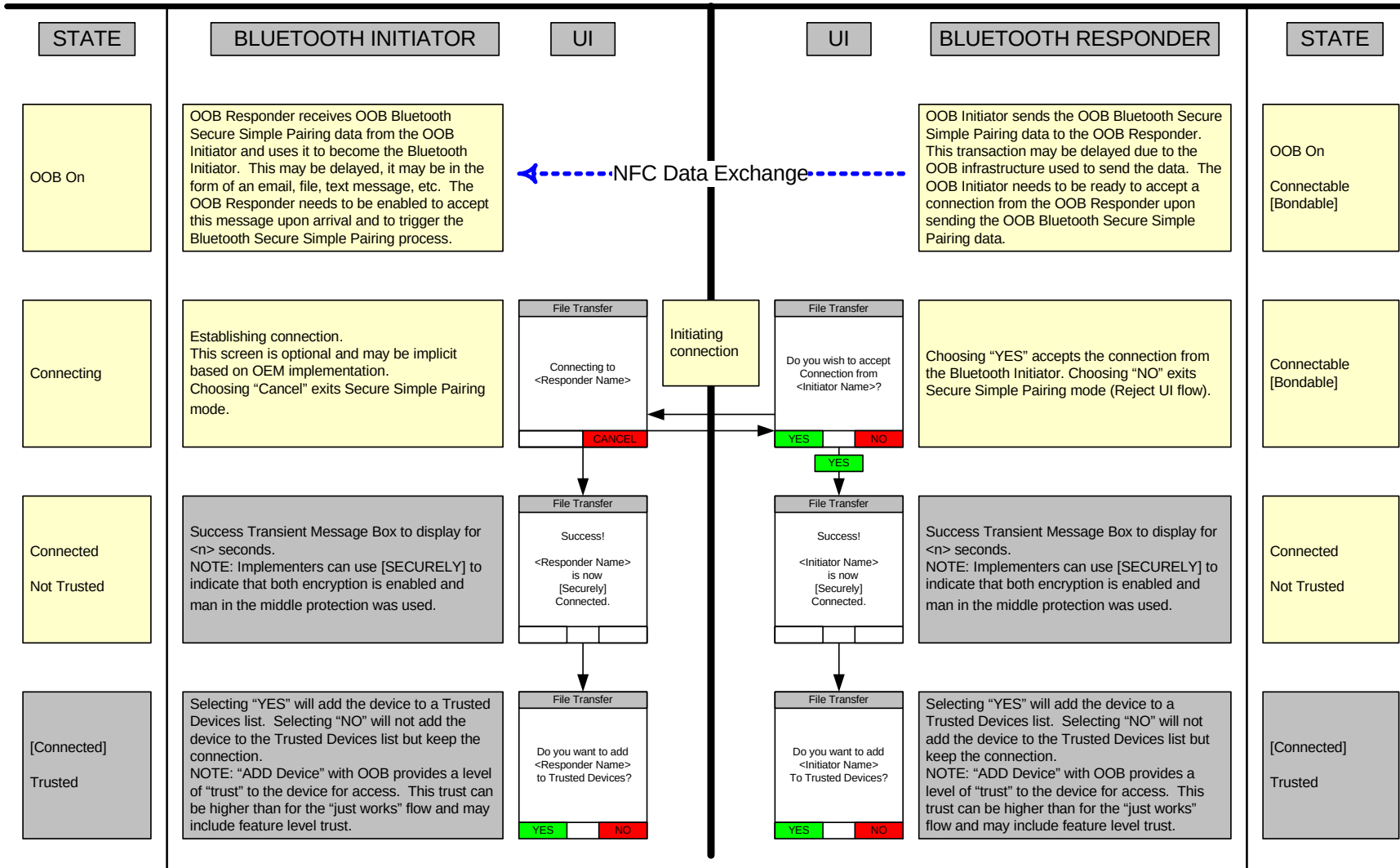


Figure 27: PDA to PDA Unidirectional OOB User Interface Flow



9.2.5.3 BIDIRECTIONAL PDA TO PDA

"OUT OF BAND" - Bidirectional PDA to PDA

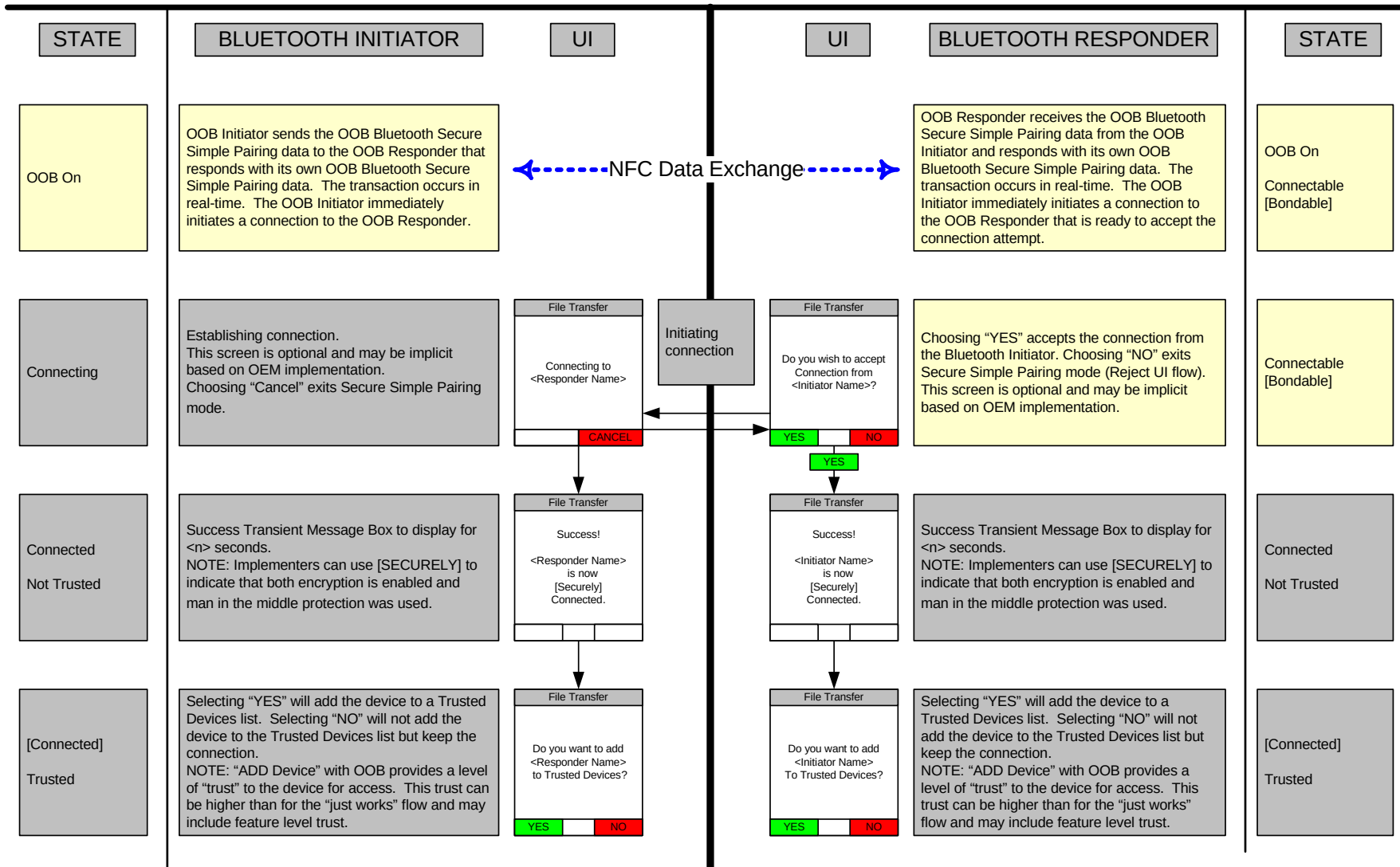


Figure 28: Out Of Band – Bidirectional PDA to PDA User Interface Flow



9.2.5.4 SEND FILE – USING NFC FROM PC TO MUSIC PLAYER

Send File – OOB using NFC from PC to Music Player

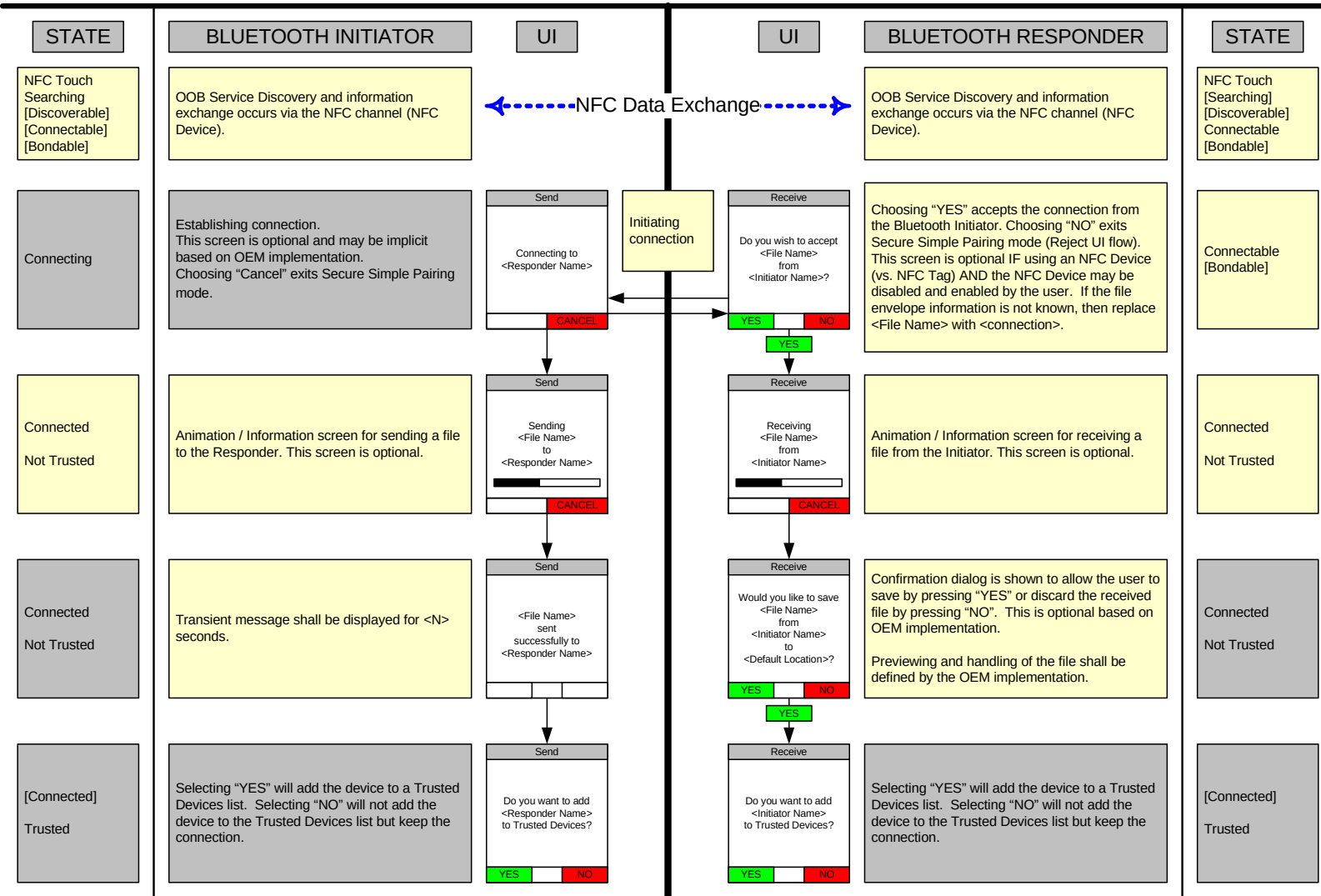


Figure 29: Send File – OOB using NFC from PC to Music Player UI Flow



9.3 CONNECTION REJECTION FROM PC TO PDA

This is used when the user decides to reject an incoming connection from a remote device. This is a new device not in the Trusted List.

Secure Simple Pairing with Connection Rejection from PC to PDA

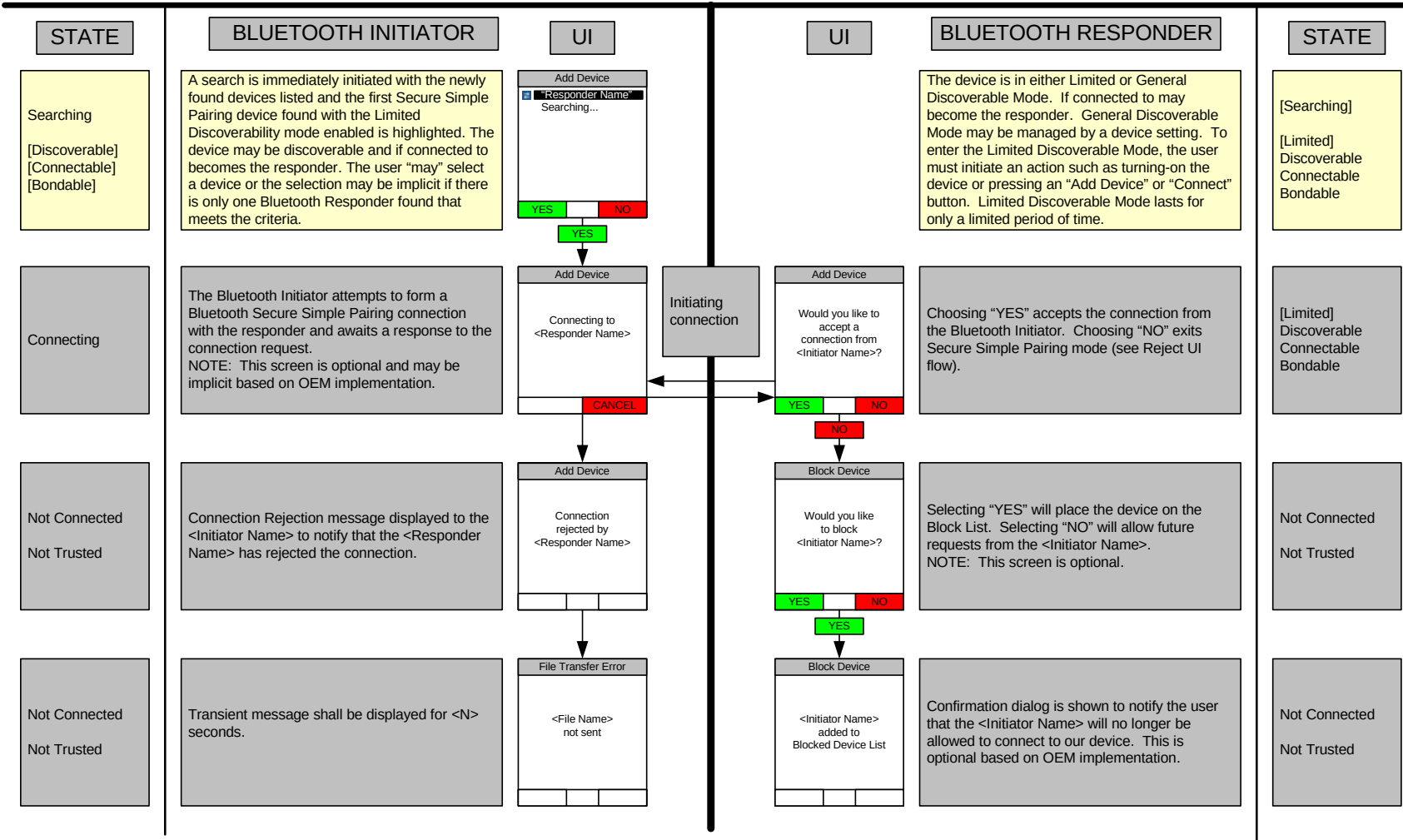


Figure 30: Secure Simple Pairing with Connection Rejection from PC to PDA UI Flow



10 Lists

These following lists are suggestive guidelines for creating lists as to what implementers may choose to support on their devices.

10.1 DEVICE LIST

This list is displayed to the user when searching for devices. This list may be sorted such that Known Devices and devices that are authenticated appear first in the list. Furthermore, the list may be filtered based upon Feature [Set]. Can be filtered such that it shows Discovered Devices or shown as Discovered List if required by the UI. It is up to the implementer to determine if the already connected devices should be shown in this list.

10.2 TRUSTED DEVICE LIST

List may be shown to the user when managing devices previously connected to the user's device. Alternate names for this list are Trusted Devices or Trusted List if required by the User Interface.

10.3 BLOCKED DEVICE LIST

If a device supports a Blocked Device List it must support a mechanism to remove a device from the Blocked Device List to allow connections to these devices.